

3.1 8086/8088 微处理器简介

1. 结构:BIU 和 EU (BIU 负责与存储器或 I/O 接口交互数据; EU 负责执行指令)

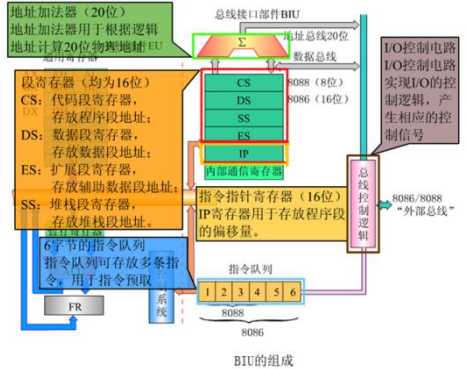
2. 总线接口单元 BIU 的组成和各功能单元的作用

功能:BIU 为 EU 完成全部的总线操作, 根据 EU 的命令控制数据在 CPU 和存储器或 I/O 接口之间传送。

操作过程 (存储器访问为例):

(1) BIU 首先将要访问存储器的逻辑地址 (CS:IP) 转换成物理地址 (PA); (2) 再从 (往) 物理地址对应的存储器单元读取 (写入) 数据; (3) 如果是读取指令, BIU 从物理地址取到指令后将指令送入指令队列。

组成:地址加法器 (地址总线 20 位); 数据总线 16 位; 指令队列 (8086 由 6 字节的寄存器组成; 8088 只有 4 字节) (队列总线 8 位); 总线控制电路 (I/O 控制电路, 产生相应控制信号); 段寄存器 (CS, DS, SS, ES, IP) 8086 为 8 位, 8086 为 16 位。



3. 执行单元 EU 的组成和各功能单元的作用

功能:负责指令的译码、执行并回写结果。此外还具有管理寄存器等功能。

工作过程:EU 从 BIU 的指令队列里得到指令后, 完成指令的译码、执行, 当执行指令的结果或执行指令所需操作数需要从存储器或者外设存取时, 便申请 BIU 访问存储器或者外设, 并向 BIU 提供段偏移地址。

组成: (1) 通用寄存器: (均为 16 位寄存器, 可以分别作为 2 个 8 位寄存器使用)

AX: 用作累加器; BX: 多用基址寄存器

CX: 多作为计数器; DX: 辅助累加器

(2) 专用寄存器 (16 位): (BP 和 SP 分别存放地址偏移量)

SP: 堆栈指针寄存器; BP: 基址指针寄存器

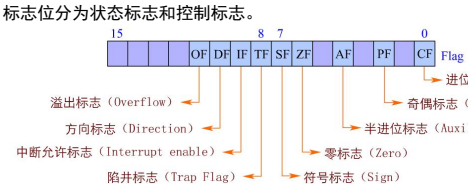
SI: 源变址寄存器; DI: 目的变址寄存

(3) 算术逻辑单元 ALU (寻址单元偏移量) (4) 执行单元 EU 的控制电路 (接收 BIU 指令队列中的指令, 并进行译码)

(5) 标志寄存器 (16 位):

其中 9 位为 8086 的标志位, 其他 7 位未用。

标志位分为状态标志和控制标志。



A. 状态标志:

CF: CF=1, 表示进位/借位; 循环指令也会影响

PF: PF=1, 低 8 位偶数个 1; PF=0, 奇数个 1

AF: AF=1, D3 位向 D4 位有进位/借位 (只有 DAA 和 DAS 指令测试 AF, 作为在 BCD 码运算时是否进行十进制调整的依据)

ZF: ZF=1, 结果为 0; ZF=0, 结果不为 0

SF: 和运算结果的最高位相同。当数据用补码表示时, 负数的最高位为 1, 所以符号标志指出了所执行运算结果的正负。

OF: OF=1, 有符号数进行加法或减法过程中产生溢出; 无符号操作不考虑该标志。

B. 控制标志: (指示 CPU 控制某种特定的功能, 通常通过指令来设定与清除)

TF: 跟踪标志或称单步标志。TF=1, 单步执行指令

IF: 对可屏蔽中断的控制标志。IF=1, 表示 CPU 对可屏蔽中断的中断允许; IF=0, 表示 CPU 不能接受可屏蔽中断请求。

由 STI (置 IF 为 1) 和 CLI (清除 IF 标志) 指令来控制。

DF: 控制串操作指令执行过程中地址的增量或减量。DF=0, 串操作过程中, 地址增值; DF=1, 串操作过程中, 地址减值。

由 STD (置位方向) 和 CLD (清除方向) 指令控制。

4. BIU 和 EU 的协同工作过程

BIU 和 EU 的工作原则:

(1) BIU 中的指令队列有 2 个或 2 个以上字节为空时, BIU 自动启动总线周期, 从存储单元取出指令填充指令队列。直至填满, BIU 才进入空闲状态。

(2) EU 每执行完一条指令, 从 BIU 指令队列的队首取指令。指令队列为空时, EU 需要等待 BIU 从内存取指令填充指令队列。

(3) EU 从指令队列取得指令后, 译码并执行指令。若该指令需访问存储器或 I/O 以取操作数或存操作结果时, EU 向 BIU 发出访问总线的请求。

(4) 当 BIU 接到 EU 申请总线的请求, 若 BIU 正忙 (正在执行取指令的总线周期), 则必须等待 BIU 执行完当前的总线周期, 方能响应 EU 请求; 若 BIU 空闲, 则立即执行 EU 的请求。

(5) EU 执行转移、调用和返回指令时, 若下一条指令不在指令队列中, 则队列中的指令被自动清除, BIU 根据转移、调用和返回指令指示的目标地址重新取出指令并填充指令队列。

指令流水:

(1) 8 位机的串行工作过程为: 从内存取指 → 对指令译码 →

读取内存中操作数 (指令需要操作数时) → 执行指令 → 回写执行结果 (必要时再次访问存储器)。

(2) 8086 中, 指令的提取和执行是分别由 BIU 和 EU 完成的, 使得 8086 可以在执行指令的同时进行读取指令的操作。

(3) 一般情况下, EU 可以不断地从指令队列中取得指令并执行指令, 与 BIU 对存储器的访问并行执行。只有在遇到系统复位或执行转移指令等特殊情况下, 指令队列被刷新时, EU 需要等待 BIU 进行取指操作。或者在 EU 需要操作数而 BIU 正忙时, EU 需要等待 BIU 执行完当前的操作, 再去取操作数, 等到 EU 得到操作数以后, 才能进行这条指令的执行操作。

5. 总线周期

CPU 的基本定时单位称为时钟周期或者状态周期。

进行一次数据传送 (访问存储器或 I/O 端口) 的总线操作定义为一个总线周期。典型的总线周期通常由 4 个时钟 (状态) 周期 T1、T2、T3 和 T4 组成。

T1: 为地址周期。复用总线上为地址信息 (通过地址/数据或地址/状态), 指示要寻址的存储器单元或者 I/O 的地址。

T2: 为缓冲周期。从总线上撤销地址 (处于悬空状态), 为数据传送做准备

T3: 为数据周期。复用总线上为数据信息。如果外设或存储器没有准备好, CPU 会在 T3 周期后插入等待周期 TW。 (信号状态和 T3 相同)

T4: 总线周期结束。CPU 采样数据总线, 完成读/写操作

3.2 8086/8088 管脚说明和工作方式

1. 信号引脚及功能

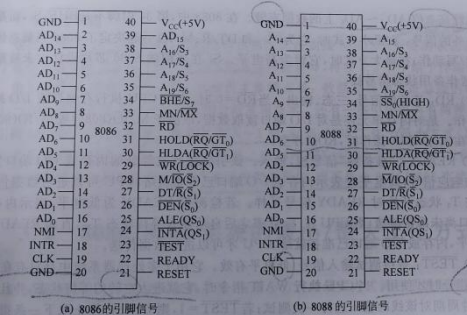


图 3.6 8086/8088 的引脚信号 (括号中为最大方式时的引脚名称)

地址/数据信号 AD15~AD0:

地址/数据时分复用信号 (2~16、39), 双向, 三态。在 T1、(地址周期) AD15~AD0 上出现的是低 16 位的地址信号 A15~A0; 在 T3 (数据周期) AD15~AD0 上出现的是数据信号 D15~D0。 (8088 中, 只传输 8 位数据, A15~A8 只传输地址)

地址/状态信号 A19/S6~A16/S3:

地址/状态复用信号 (35~38), 输出。在 T1 状态 (地址周期) A19/S6~A16/S3 上出现的是地址的高 4 位。在 T2~T4 状态, A19/S6~A16/S3 上输出状态信息。

S6: 指示 8086 当前是否与总线相连。S6=0, 8086 连在总线上。

S5: 表示中断允许标志状态。S5=1 时, 中断允许标志 IF=1; S5=0 表示 IF=0, 禁止可屏蔽中断。

S4 和 S3 用来指出当前使用的段寄存器。S4、S3 代码组合对应的含义如表所示:

S ₄	S ₃	当前正在使用的段寄存器
0	0	ES
0	1	SS
1	0	CS 或未使用任何段寄存器
1	1	DS

控制总线信号:

(1) BHE*/S7: 数据信号高 8 位使能 and 状态复用信号 (34), 在 T1 状态, BHE*有效, 表示数据线上高 8 位数据是有效的。在 T2~T4 状态, BHE*/S7 输出状态信息 S7 (S7 未实际定义)。

(2) RD*: 读信号 (32), 输出, 三态, 低电平有效。RD*=0, 表示 CPU 执行一个对存储器或 I/O 端口的读操作, 在一个读操作的总线周期中, RD*在 T2~T3 状态中有效, 为低电平。

(3) READY: 准备好信号 (22), 输入, 高电平有效。CPU 在 T3 状态对 READY 进行采样。READY=1, 表示存储器或 I/O 准备好发送或接收数据。READY=0, 在 T3 之后插入 TW, 使 CPU 能和较慢速度的存储器或 I/O 接口相匹配。

(4) TEST*: 测试信号 (23), 输入, 低电平有效。TEST*信号和 WAIT 指令令使用。CPU 执行 WAIT 指令时, 处于空转状态, 进行等待。只有当 8086 检测到 TEST*=0 时, 才结束等待状态, 继续执行 WAIT 之后的指令。

(5) NMI: 非屏蔽中断请求 (17), 输入, 上升沿有效。NMI 不受中断允许标志 IF 的影响。NMI 有效时, CPU 执行完当前指令便响应中断类型为 2 的非屏蔽中断请求。

(6) INTR: 可屏蔽中断请求 (18), 输入, 高电平有效。INTR=1, 且 IF=1 时, CPU 结束当前指令后, 响应 INTR 中断。

(7) RESET: 复位信号 (21), 输入, 高电平有效。RESET=1 时, CPU 结束当前操作并对标志寄存器 FLAG、IP、DS、SS、ES 及指令队列清零, 并将 CS=FFFFH。当 RESET 撤除时, CPU 从 FFFF0H 开始执行程序。

(8) CLK: 时钟信号 (19), 输入。为 CPU 和总线控制逻辑提供定时。要求时钟信号的占空比为 33% (低: 高=2: 1)。

电源线 VCC (5V+/-10%) 和两条地线 GND (1, 20)

MM/MX*: 最小/最大工作方式选择信号 (33)

2. 最小工作方式 (单处理器系统方式)

MM/MX*=1 (接电源电压), 此时全部信号由 CPU 本身提供

(1) INTA*: 中断响应信号 (24), 输出, 低电平有效。用于外设对中断请求作出响应。在 8086/8088 系统中, 该信号为两个连续的电脉冲, 第一个表示 CPU 接收了外设的中断请求信号, 第二个表示 CPU 要求外设通过数据总线提供该中断的类型号。

(2) ALE: 地址锁存使能信号 (25), 输出, 高电平有效。用来作为地址锁存器的锁存控制信号。8086 的 AD15~AD0 是地址

/数据复用信号, 地址信息仅在 T1 状态有效, 为使地址信号在整个读写周期有效, 用 ALE 把地址信号锁存在地址锁存器中。

(3) DEN*: 数据使能信号 (26), 输出, 三态, 低电平有效。用于数据总线驱动器的控制信号。

(4) DT/R*: 数据驱动器数据流向控制信号 (27), 输出, 三态。在 8086 中, 常采用 8286 或 8287 为数据总线的驱动器, DT/R*控制驱动器的数据传送方向。DT/R*=1, CPU 通过数据驱动器进行数据发送; DT/R*=0, 通过数据驱动器进行数据接收。

(5) M/IO*: 存储器或 I/O 控制信号 (28), 输出, 三态。M/IO*=1, 指示 CPU 正在执行存储器访问指令, 进行和存储器之间数据交互; M/IO*=0, 表示 CPU 正在进行和 I/O 接口之间数据传输。

(6) WR*: 写信号 (29), 输出, 三态。WR*信号有效, 表示 CPU 执行一个对存储器或 I/O 端口写操作, 在写操作总线周期中, WR*在 T2~T3 状态中有效, 为低电平。

(7) HOLD: 总线保持请求 (31), 输入, 高电平有效。当系统中另一个总线主模块 (如 DMA) (除 CPU) 要求使用总线时, 该主模块通过 HOLD 信号向 CPU 发出总线请求。若 CPU 允许让出总线, 就在完成当前总线周期后, 在 T4 状态通过 HLDA 引脚发出应答信号, 响应总线的请求。

(8) HLDA: 总线保持响应信号, 输出, 高电平有效。HLDA=1, 则 CPU 响应了其他主模块的总线请求, CPU 的数据/地址控制信号变为高阻状态, 而请求总线的主模块 (DMA) 获得了总线权。

3. 最大工作方式 (多处理器系统方式)

MM/MX*=0, 控制信号由 8288 总线控制器和 CPU 联合提供

(1) QS1, QS0: 指令队列状态信号 (25, 24), 输出。QS1, QS0 组合表示前一个时钟周期中指令队列的状态。使得可以从外部对 8086 指令队列进行跟踪。用于对芯片的测试。

QS1=0 QS0=0: 无操作; QS1=1 QS0=0: 队列为空

QS1=0 QS0=1: 从指令队列的第一个字节中取走代码; QS1=1 QS0=1: 除第一个字节外, 取后两字节的代码。

(2) S2*, S1*, S0*: 总线周期状态信号 (28、27、26), 输出。组合表示当前执行的总线周期的类型。在最大模式下, 用这三个信号作为总线控制器 8288 的输入, 产生存储器、I/O 的读、写等控制信号。S2*, S1*, S0* 的组合意义:

S2*	S1*	S0*	总线周期的类型
1	1	1	无源状态
1	1	0	写内存
1	0	1	读内存
1	0	0	取指
0	1	1	暂停
0	1	0	写 I/O
0	0	1	读 I/O
0	0	0	中断响应

(3) LOCK*: 总线封锁信号 (29), 输出。用来封锁其它总线主部件的总线请求。LOCK*=0, 系统中其他总线主部件不能占用总线。LOCK*信号是由前级 LOCK 产生的。在 LOCK 前级后的指令执行完之后, 硬件上便撤销了 LOCK*信号。

(4) RQ*/GT1*, RQ*/GT0*: 总线请求信号/总线请求允许信号 (31、30), 双向。CPU 以外的两个处理器可以分别用其中之一来请求总线并接受 CPU 对总线请求的允许。其中 RQ*/GT0*优先级高于 RQ*/GT1*。

3.3 8086/8088 的存储器

1. 存储单元的地址和内容

常用数据类型:

字节: 8 位 (存储器的基本单位); 字: 16 位

双字: 32 位 (4 字节); 四字: 64 位 (8 字节)

物理地址: 20 位地址码, 存储器译码后可选定存储单元

偏移地址: 相对于首地址的段内偏移量

逻辑地址: 写作: 段地址: 偏移地址

2. 存储器的组织方式

寻址空间: 2²⁰=1MB 的存储器寻址空间

分段组织: 各段首地址保存在 (CS、DS、SS、ES) 中, 各段可以继续, 分开或重叠 (最大 64KB 的连续存储空间)

段地址 × 16d + 偏移地址 = 物理地址 (20bit)

字与字节访问:

低位字节 → 奇数地址 → 非规则 → 两个总线周期

低位字节 → 偶数地址 → 规则 → 一个总线周期

8086 的 1MB 存储空间分为两个 512KB 存储库: 高位库和低位库

低位库: 数据总线 D7~D0 相连, 每个地址为偶数地址

高位库: 数据总线 D15~D8 相连, 每个地址为奇数地址

AD0 和 BHE*组合, 选择奇偶字节或字, 实现对存储器的访问:

BHE*=0, AD0=0: 同时读/写高、低两个字节 (AD15~AD0)

BHE*=1, AD0=0: 只读/写偶数地址低位字节 (AD7~AD0)

BHE*=0, AD0=1: 只读/写奇数地址高位字节 (AD15~AD8)

BHE*=1, AD0=1: 不传送

3. 堆栈 (用于存放数据的一个区域)

“先进后出, 后进先出”的存放和取用数据的方式

栈顶: 是堆栈操作的唯一出口 (栈顶 SP 小, 栈底 BP 大)

入栈指令 PUSH: 堆栈指针 SP 每执行一次 PUSH, SP-2

出栈指令 POP: 每执行一次 POP, SP+2

3.4 8086/8088 的指令系统

1. 寻址方式 (指令基本格式为操作码+操作数)

固定指令: 某些单字节指令中使用, 该指令是 CPU 针对某个固定的寄存器操作的, 操作数位于规定的寄存器中。例: AAA

立即数寻址: 操作数包含在指令中, 是一个 16/8 位的常数, 称为立即数。例: MOV AL, 5; MOV AL, 'A'

注: 立即数只能用于源操作数

寄存器寻址: 操作数包含在寄存器中, 由指令指定寄存器名。例: MOV AL, BH **注: 字节寄存器只有 (AH, BH, CH, DH, DL)**

存储器寻址: 操作数在存储器中, 偏移地址在指令中常用方括号 [] 或符号地址表示。

(1) 直接寻址: 操作数的有效地址 (偏移量) EA 在指令中直接给出。例: MOV AX, [2000H] 则 EA=2000H, 假设 (DS)=3000H, 可得 PA=32000H **注: 1. 隐含的段为数据段 (DS) 2. 如不在数据段中, 可以用段跨越前缀, 例: MOV AX, ES: [2000H] 3. 操作数地址可以由变量 (符号地址) 表示, 例: MOV AX, COUNT**

(2) **间接寻址**:操作数的有效地址(偏移量)EA 与寄存器(基址和变址) 有关

A. 基址寻址:有效地址由基址寄存器(BX/BP)和指令中的偏移量给出。例: MOV AX, [BX+1000H]; 则 PA=16d×(DS)+(BX+1000H)

注: 不允许使用 AX CX DX

B. 变址寻址:有效地址由变址寄存器(SI/DI)和指令中的偏移量给出。例: MOV AX, [DI]; 则 PA=16d×(DS)+(DI) **注: MOV AX, COUNT[BX][SI]等价于 MOV AX, COUNT[BX+SI]等价于 MOV AX, [COUNT+BX+SI]**

C. 基址加变址寻址:

偏移地址=(BX)/(BP)+(SI)/(DI)+8 位/16 位偏移量

其他寻址方式:

(1) **串操作指令寻址**:用于数据串指令。源串操作数第一个字节或字的有效地址存放在源变址寄存器 SI 中, 目标串操作数第一个字节或字的有效地址存放在目标变址寄存器 DI 中。

(2) **I/O 端口寻址**:IN/OUT

A. 直接端口寻址:端口号在指令中以 8 位立即数方式直接给出, 其范围是 00~FFH

B. 间接端口寻址:端口号在 DX 寄存器中给出, 范围 0000~FFFFH

C. 转移类指令寻址:段内/段间+ 直接/间接转移(JMP 指令)

2. 指令分类

(一) **数据传送指令**

(1) **通用数据传送指令**

MOV d, s; 传送指令, d←s

1. 目标操作数 d 和源操作数 s 不能同时用存储器寻址方式;(适用于所有双操作数指令)

2. SRC 和 DST 的字节长度一致; MOV BX, AL ×

3. 目标操作数 d 不能是 CS, 也不能为立即数; MOV CS, AX ×

4. 立即数不能直接送到段寄存器; MOV ES, 2000H ×

5. d 和 s 不允许同时为段寄存器; MOV ES, DS ×

6. MOV 指令不影响标志位。

PUSH s; 低字节存入栈顶, (SP)←(SP)-2, ((SP))←(s)

POP d; 栈顶值移入低字节, (d)←((SP)), (SP)←(SP)+2

1. PUSH 和 POP 指令只能字操作, SP 的修改必须是 +2 或-2;

2. PUSH 和 POP 指令不能使用立即数寻址方式;

3. POP 指令的 DST 不允许是 CS 寄存器, PUSH 可以;

4. PUSH 和 POP 指令都不影响标志位。

XCHG d, s; 交换指令, d↔s

1. XCHG 指令使两个操作数其中一个操作数必须在寄存器中, 另一个操作数可以在寄存器或存储器中; 例: XCHG BX, [BP+SI]

2. 不允许使用段寄存器和 IP;

3. 不影响标志位。

XLAT; 换码指令, (AL)←((DS)×16+(BX)+(AL))

先建立字节表格, 其首地址放到 BX 中。指令根据 AL 寄存器提供的位移量, 将 BX 指示的字节表格中的代码换存在 AL 中。该指令还可写为: XLAT opr. opr 为字节表格的首地址, 因为其表示的偏移地址已存入 BX, 所以 opr 可有可无, 有则可提高程序的可读性。例: 将表格 TABLE 中位移量为 3 的代码取到 AL:

DS=FO0H			
TABLE		30H	F000H
(BX)←	31H		F001H
(AL)←	32H		F002H
	33H		F003H

TABLE	30H	MOV BX,OFFSET TABLE ;(BX)←offset of TABLE
	31H	MOV AL,3 ;(AL)←displacement
	32H	XLAT TABLE ;(AL)←(BX)+(AL)=33H
	33H	
	34H	

1. 字节表格长度≤256 字节 (因为 AL 为 8 位寄存器) ;

2. 不影响标志位。

(2) **目标地址传送指令**

LEA reg, s; reg←offset of s (有效地址送至 16 位通用寄存器) 例: · LEA AX, [0618H]; 即 AX←0618H

注: (LEA 和 MOV 的区别) LEA 传送 s 指示的偏移地址, MOV 传送 s 寻址的存储单元的数据;

LDS reg, s; reg←s 且 DS←s+2

(偏移地址送寄存器和段地址送 DS)

LES reg, s; reg←s 且 ES←s+2

(偏移地址送寄存器和段地址送 ES)

例: LES DI, [BX];

设当前 DS=B000H, BX=080AH, B080AH 单元存储 05A2H, B080CH 单元存储 4000H, 则该指令将 05A2H 送入 DI, 4000H 送入 ES

1. 指令中的 reg 不能是段寄存器 (ES DS SS CS) ;

2. 指令中的 s 必须使用存储器寻址方式;

3. 不影响标志位。

(3) **标志寄存器传送指令**

LAHF; 标志寄存器的低字节送 AH, (AH)←(FLAGS) 0-7

SAHF; AH 送标志寄存器低字节, (FLAGS) 0-7←(AH)

PUSHF; 标志进栈, ((SP)+1, (SP))←(FLAGS) 0-15; SP←SP-2

POPF; 标志出栈, (FLAGS) 0-15←((SP)+1, (SP)); SP←SP+2

1. LAHF 和 SAHF 指令隐含的操作寄存器是 AH 和 FLAGS;

2. LAHF 和 PUSHF 不影响标志位, SAHF 和 POPF 由装入的值确定。

(4) **I/O 数据传送**

IN AL/AX (累加器), port (端口号); port←0FFH

(AL)←(port)传送字节; (AX)←(port+1, port)传送字

IN AL/AX, DX; 0≤DX 中的 port≤0FFFFH

(AL)←((DX))传送字节; (AX)←((DX)+1, (DX))传送字

OUT port, AL/AX; (AL)←(port)或 (AX)←(port+1, port)

OUT DX, AL/AX; (AL)←((DX))或 (AX)←((DX)+1, (DX))

1. 只限于在 AL 或 AX 与 I/O 端口之间传送信息;

2. 不影响标志位。

(二) **算术运算指令**

(1) **加法指令**

ADD d, s; d←s+d

1. 目标操作数 d 不允许是立即数;

2. 源和目标操作数不能同时为存储器操作数;

3. 影响状态标志位。

ADC d, s; d←s+rd+OF

1. 目标操作数 d 不允许是立即数;

2. 源和目标操作数不能同时为存储器操作数;

3. 影响状态标志位, CF 置为新状态。

INC d; 加 1 指令, d←d+1

1. 将目标操作数当作无符号数;

2. 对于间接寻址, 数据必须用 BYTE PTR、WORD PTR 或 DWORD PTR 类型的伪指令来说明, 以确定是对字节、字还是双字加 1;

3. 不影响进位标志 CF, 影响其它 5 位状态标志位。

(2) **减法指令**

SUB d, s; d←d-s

1. 目标操作数 d 不允许是立即数;

2. 源和目标操作数不能同时为存储器操作数;

3. 影响状态标志位。

SBB d, s; d←d-s-CF

1. 目标操作数 d 不允许是立即数;

2. 源和目标操作数不能同时为存储器操作数;

3. 影响状态标志, CF 置为新状态。

DEC d; d←d-1

1. 目的操作数 d 不允许是立即数;

2. 影响状态标志位, 不影响 CF;

3. 对于间接寻址, 数据必须用 BYTE PTR、WORD PTR 或 DWORD PTR 类型的伪指令来说明, 以确定是对字节、字还是双字减 1。

NEG d; d←-d, 按位取反后保持符号位不变加 1 (带符号数)

例: 假设 AL=00000100=4, NEG AL 后, AL=11111100=[-4]补

假设 AL=11101110=[-18]补, NEG AL 后, AL=00010010=18

注: NEG 指令的标志位码设置方法为:

不为 0 的操作数求补时, CF=1; 为 0 的操作数求补时, CF=0;

(原理: 实际上是用 0 减去, 一般会产生借位)

当求补运算的操作数为-128 字节或-32768 字时, OF=1;

当求补运算的操作数不为-128 字节或-32768 字时, OF=0

CMP d, s; d-s, 不送回结果, 只根据运算结果置标志位

两无符号数: 若 ZF=1, 则 d=s; 若 ZF≠1, 则 d≠s;

若 CF=0, 则 d≥s; 若 CF=1, 则 d<s;

两有符号数: OF⊕SF=0, d≥s; OF⊕SF=1, d<s

1. 目标操作数 d 不允许是立即数;

2. 源和目标操作数不能同时为存储器操作数;

3. 影响状态标志位。

(3) **乘法指令**

MUL s; 无符号数乘法

字节操作: (AX)←(AL)×(s); 字操作: (DX, AX)←(AX)×(s)

1. AL (AX) 为隐含的被乘数寄存器; 2. AX (DX, AX) 为隐含的乘积寄存器; 3. s 不能为立即数; 4. 除 CF 和 OF 外, 对其它状态标志位无意义。

注: MUL 对标志位的影响:

若高位字 DX 或高位字节 AH≠0, 则 CF=OF=1, 否则, CF=OF=0

IMUL s; 带符号数乘法

字节操作: (AX)←(AL)×(s); 字操作: (DX, AX)←(AX)×(s)

1. AL (AX) 为隐含的乘数寄存器;

2. AX (DX, AX) 为隐含的乘积寄存器;

3. SRC 不能为立即数;

4. 除 CF 和 OF 外, 对其它状态标志位无意义。

注: IMUL 对标志位的影响:

若乘积高一半为低一半的符号扩展, 则 CF=OF=0, 否则, CF=OF=1

(4) **除法指令**

DIV s; 无符号数除法

字节操作: (AL)←(AX)/s 的商; (AH)←(AX)/s 的余数

字操作: (AX)←(DX, AX)/s 的商; (DX)←(DX, AX)/s 的余数

IDIV s; 带符号数除法

执行操作与 DIV 相同, 但操作数必须是带符号数, 商和余数也均为带符号数, 而且余数的符号与被除数的符号相同。

1. AX (DX, AX) 为隐含的被除数寄存器;

2. AL (AX) 为隐含的商数寄存器;

3. AH (DX) 为隐含的余数寄存器;

4. SRC 不能为立即数;

5. 对所有状态标志位均无定义;

6. 除法指令要求字操作时, 被除数必须为 32 位, 除数是 16 位, 商和余数是 16 位的; 字节操作时, 被除数必须为 16 位, 除数是 8 位, 得到的商和余数是 8 位的;

7. 除数为 0 或商溢出等错误, 由系统直接转入 0 型中断来处理。商溢出, 指被除数高一半的绝对值大于除数的绝对值时, 商超出了 16 位的表示范围 (字操作) 或 8 位 (字节操作)。

(5) **符号扩展指令**

CBW; 字节扩展为字

AL 最高有效位为 0 时, AH=00H; AL 最高有效位为 1 时, AH=FFH

CWD; 字扩展为双字

AX 最高有效位为 0 时 DX=0000H; AX 最高有效位为 1 时 DX=FFFFH

1. 进行符号扩展的操作数必须存放在 AL 寄存器或 AX 寄存器中;

2. 不影响状态标志位。

注: 除法指令要求字操作时, 被除数必须为 32 位; 字节操作时, 被除数必须为 16 位。可用上述两指令对带符号数除法的被除数进行扩展。

(6) **十进制调整指令**

压缩的 BCD 码:用 4 位二进制数表示一个十进制数位。

非压缩的 BCD 码:用 8 位二进制数表示一个十进制数位, 其中低 4 位是 BCD 码, 高 4 位是 0。

把 ASCII 码的高 4 位清零即得 BCD 码 (可表示 128 种字符)

一般使用 AND AL, 0FH 进行转换

A. 压缩的 BCD 码调整指令:

DAA; 加法的十进制调整:把 AL 中的和调整为压缩的 BCD 格式

DAS; 减法的十进制调整:把 AL 中的差调整为压缩的 BCD 格式

DAA 和 DAS 指令的调整方法如下:

执行加法指令或减法指令后,

1. 结果低 4 位 (AL) D0-D3>9 或 AF=1, 则 AL←AL±06H, 且 AF=1;

2. 结果高 4 位 (AL) D4-D7>9 或 CF=1, 则 AL←AL±60H, 且 CF=1。

注: 这两个调整的条件, 如果满足其一, 则±06H 或±60H; 若同时满足, 则±06H 后, 再±60H。

B. 非压缩的 BCD 码调整指令:

AAA; 加法的 ASCII 调整

AL←把 AL 中的一位 ASCII 码数加法运算的结果调整为非压缩的 BCD 格式; AH←AH+调整产生的进位值

AAS; 减法的 ASCII 调整

AL←把 AL 中的 ASCII 码数减法算的结果调整为非压缩的 BCD 格式; AH←AH-调整产生的借位值

AAA 和 AAS 的调整方法如下:

执行加法指令或减法指令后,

1. 若 (AL) D0-D3=0~9, 且 AF=0, 则 (AL) D4-D7=0, CF=0;

2. 若 (AL) D0-D3=A~F, 或 AF=1, 则 AL←AL±06H, (AL) D4-D7=0, (AH)←(AH)±1, AF 的值送 CF。

注: 1. AAA 和 AAS 指令只影响 AF 和 CF, 其余无定义

2. AAA 和 AAS 指令是专门为 ASCII 码操作数或非压缩 BCD 码操作数的加法而设计的。

AAM; 乘法的 ASCII 调整

AX←把 AX 中的积调整为非压缩的 BCD 格式

注: 将非压缩 BCD 码相乘后的运算结果转换为非压缩的 BCD 码 AAM 的调整方法为:

AH←AL/OAH 的商; AL←AL/OAH 的余数

1. AAM 实际上是将 AL 中的二进制数进行二化十的转换, 十位数放在 AH 中, 个位数放在 AL 中;

2. 若两个 ASCII 码数相乘, 要先将它们转换成非压缩 BCD 码;

3. AAM 只对 AX 寄存器中的数进行调整, 只影响 SF、ZF 和 PF 标志位, 其它无定义。

AAD; 除法的 ASCII 调整执行操作:

AX←AX 中的被除数 (非压缩的 BCD 格式) 转化为二进制数。

AD 的调整方法为:

执行除法指令之前, 对 AX 中的非压缩 BCD 码 (被除数) 执行: AL←AH×10+AL; AH←0

注: 与其它调整指令不同的是, AAD 用在 DIV 指令之前, 即将 AX 中的被除数调整成二进制数, 并存放在 AL 中, 再用 DIV 指令作二进制数的除法。

1. AAD 实际上是对 AX 中的十进制数进行十化二的转换;

2. AAD 只对 AX 寄存器中的数进行调整, 只影响 SF、ZF 和 PF 标志位, 其它无定义。

(三) **逻辑运算和移位循环指令**

(1) **逻辑运算指令**

AND d, s; 逻辑与; OR d, s; 逻辑或

NOT d; 逻辑非; XOR d, s; 异或

TEST d, s; 测试, 按 AND 操作, 不返回结果, 影响标志位

1. 逻辑运算指令是一组位操作指令, 它们可以对字节或按位执行逻辑操作, 因此, 源操作数经常是一个位串;

2. 以上五条指令除 NOT 不影响标志位外, 其它四条指令执行后, CF=OF=0 (TEST 的 CF 也无定义), AF 无定义, SF、ZF 和 PF 根据运算结果设置;

3. s 可以用寄存器, 存储器操作数或立即数, d 不允许立即数。

(2) **移位指令和循环移位指令**

SHL d, c; 逻辑左移, 最低位补 0, 最高位移入 CF

SHR d, c; 逻辑右移, 最高位补 0, 最低位移入 CF

SAL d, c; 算术左移, 最低位补 0, 最高位移入 CF

SAR d, c; 算术右移, 符号位的值补最高位, 最低位移入 CF

ROL d, c; 循环左移, 最高位移入 CF 和最低位

ROR d, c; 循环右移, 最低位移入 CF 和最高位

RCL d, c; 带进位循环左移, CF 移入最低位, 最高位移入 CF

RCR d, c; 带进位循环右移, CF 移入最高位, 最低位移入 CF

注: 1. 移位次数 c=1 时, 可以直接写, c>1 时, 必须先放入 CL 2. c=1, 且符号位发生变化, 则 OF=1; 若 c>1, 则 OF 无效

(四) **串操作指令**

(1) **字符串指令**

MOVS (d, s); 串传送 (MOVSB 字节传送 MOVSW 字传送)

1. 将 SI 作为指针的源串中的元素传送到 DI 作为指针的目标串中 (ES:DI)←(DS:SI)

2. SI←SI±1 (字节) 或±2 (字); DI←DI±1 (字节) 或±2 (字); 自动修改指针 SI/DI, 使之指向下一个元素

CMPS (d, s); 串比较 (CMPSB/CMPSW)

1. (DS:SI)-(ES:DI), 源串中元素减去目标串中对应元素, 不送回结果, 只根据比较的结果设置状态标志位

2. 自动修改指针 SI/DI, 使之指向下一个元素

SCAS (d); 串搜索 (SCASB/SCASW)

1. (AL)-(ES:DI) 或 (AX)-(ES:DI), AX 或者 AL 中的关键字减去目标串元素, 不传送结果, 只根据比较的结果设置状态标志位

2. 自动修改指针 DI, 使之指向下一个元素

LODS (s); 读串 (LODSB/LODSW)

1. (AL) 或 (AX)←(DS:SI); 将源串所指元素取入 AX/AL 寄存器

2. 自动修改指针 SI, 使之指向下一个元素

STOS (d); 写串 (STOSB/STOSW)

1. (ES:DI)←(AL) 或 (AX); 将 AX/AL 寄存器中的内容写入目标串所指元素中

2. 自动修改指针 DI, 使之指向下一个元素

注: 源串起始地址为 DS:SI, 允许使用段超越前缀修改源串地址。目的串起始地址为 ES:DI, 禁止用段超越前缀修改 ES。

(2) **重复前缀**

REP; 重复执行

（五）程序控制指令

（1）无条件转移指令

A. 段内直接转移:位移量在指令中直接给出，转移的目标地址在指令中可直接使用符号地址。

IP←IP 当前+16/8 位位移量

1. 短转移转移范围:127~—128 字节, 写为 JMP SHORT LABLE
2. 近转移, 位移量为 16 位, 转移范围是-32768 至 +32767, 可转移到段内的任一个位置. 写为 JMP NEAR LABLE 。(近转移是 JMP 指令的缺省格式, 可以写为 JMP label)

B. 段内间接转移:

JMP reg (寄存器间接转移) IP←reg 指令
JMP WORD PTR OPR (存储器间接转移) (IP) ←(PA+1, PA) 把 PA 单元的字内容送到 IP 寄存器中。例: JMP WORD PTR [DI]; PA=DS×16d+DI, 指令执行的结果是 (IP)=(PA+1, PA)

C. 段间直接转移:指令中直接给出目标标号的段基址和偏移地址。例: JMP FAR PTR label

D. 段间间接转移:目标地址的段地址和偏移地址存放于存储器的 4 个连续地址中, 前两个字节为偏移地址, 后两个字节为段地址。指令中只须给出这 4 个连续地址的首字节偏移地址。

例: MOV SI, 1000H
JMP DWORD PTR [SI]

执行后: IP←[DS:1000H], [DS:1001H]
CS←[DS:1002H], [DS:1003H]

JMP 目标标号:

1. 根据跳转地址的偏移范围分为短、近、远转移, 远转移的目标地址也可以使用除立即寻址方式外的任何寻址方式。
2. 根据操作数的寻址方式, 可以分为段内直接转移、段内间接转移、段间直接转移和段间间接转移。

CALL 过程名; 子程序调用指令

A. CALL N.PROC(子程序名) 段内直接近调用

- ① (SP) ←(SP)-2, ((SP)) ←(IP) 当前 (保存断点)
- ② (IP) ←(IP) 当前+16 位位移量 (在指令的第 2、3 个字节中)

B. CALL DESTIN 段内间接近调用

- ① (SP) ←(SP)-2, ((SP)) ←(IP) 当前 (保存断点)
- ② (IP) ←(EA); (EA) 为指令寻址方式所确定的有效地址

例: CALL BX, 结果将 BX 的内容送到 IP 中

C. CALL FAR PTR SUBROUT 段间直接远调用

- ① (SP) ←(SP)-2, ((SP)) ←(CS) 当前;
(SP) ←(SP)-2, ((SP)) ←(IP) 当前

- ② (IP) ←偏移地址 (在指令的第 2、3 个字节中), (CS) ←段地址 (在指令的第 4、5 个字节中)

D. CALL DWORD PTR DESTIN 段间间接远调用

- ① (SP) ←(SP)-2, ((SP)) ←(CS) 当前;
(SP) ←(SP)-2, ((SP)) ←(IP) 当前
- ② (IP) ←(EA); (EA) 为指令寻址方式 所确定的有效地址;
(CS) ←(EA+2)

RET; 返回指令

把保存在堆栈中的返回地址出栈, 返回到调用程序

近返回 (IP) ←((SP)), (SP) ←(SP)+2;

远返回 (IP) ←((SP)), (SP) ←(SP)+2;
(CS) ←((SP)), (SP) ←(SP)+2

RET N; 带立即数返回

- ① 返回地址出栈 (操作段内或段间返回)
- ② 修改堆栈指针: (SP) ←(SP)+N

（2）条件转移指令

指令名称	助记符	测试条件
C-2 对无符号数	高于/不低于/不等于	JA/JNBE 目标标号
	高于或等于/不低于	JAE/JNB 目标标号
	低于/不低于/不等于	JB/JNBE 目标标号
	低于或等于/不低于	JBE/JNA 目标标号
S-2, 0 对带符号数	大于/小于/不等于	JG/JNLE 目标标号
	大于或等于/不小于	JGE/JNL 目标标号
	小于/不大于/不等于	JL/JNGE 目标标号
	小于或等于/不大于	JLE/JNG 目标标号
C-2, D.P 单标志	等于/结果为零	JE/JZ 目标标号
	不等于/结果不为零	JNE/JNZ 目标标号
	有进位/有借位	JC 目标标号
	无进位/无借位	JNC 目标标号
位条件	溢出	JO 目标标号
	不溢出	JNO 目标标号
转移	奇偶性为 1/偶状态	JP/JPE 目标标号
	奇偶性为 0/奇状态	JNP/JPO 目标标号
	符号位为 1	JS 目标标号
	符号位为 0	JNS 目标标号

注: 所有条件转移指令都是 (段内直接转移中的) 短转移指令, 转移的目标地址必须当前 IP 地址的-128 至 +127 字节范围之内, 因此条件转移指令是 2 字节指令
(3) 循环控制指令 (CX 为循环次数)

LOOP label 循环

① (CX) ←(CX)-1; ②若 (CX) ≠0, 则 (IP) ←(IP) 当前+位移量, 实行循环, 否则循环结束

LOOPZ/LOOPE label 为零/相等时循环

① (CX) ←(CX)-1; ②若 ZF=1 且 (CX) ≠0, 则 (IP) ←(IP) 当前+位移量, 实行循环, 否则循环结束

LOOPNZ/LOOPE label 不为零/不等时循环

① (CX) ←(CX)-1; ②若 ZF=0 且 (CX) ≠0, 则 (IP) ←(IP) 当前+位移量, 实行循环, 否则循环结束

注: 1. CX 中存放循环次数; 2. 只能使用段内直接寻址的 8 位位移量, 范围在-128~+127 字节之间

JCXZ label: 若 (CX) =0, 则转移

注: 该指令不修改 CX 的值, 只根据 CX 的内容转移

（4）中断指令

INT n; 中断指令, n 为中断类型号

- ① 入栈保存 FLAGS: (SP) ←(SP)-2, ((SP)) ←(FLAGS)
- ② 入栈保存返回地址: (SP) ←(SP)-2, ((SP)) ←(CS);
(SP) ←(SP)-2, ((SP)) ←(IP)

③ 转中断处理程序: (IP) ←(n×4), (CS) ←(n×4+2)

INT0; 溢出时中断, 相当于 INT 4

若 OF=1 (有溢出), 则 ①②③ 转中断处理程序: (IP) ←(4×4)= (10H), (CS) ←(4×4+2)= (12H)

注: 1. 隐含的中断类型为 3, 即 INT 等价于 INT 3;

2. INT 指令 (包括 INTO) 执行后, 使 IF=TF=0, 但不影响其它标志位

IRET; 中断返回指令

- ① 返回地址出栈: (IP) ←((SP)), (SP) ←(SP)+2;
(CS) ←((SP)), (SP) ←(SP)+2

② FLAGS 出栈: (FLAGS) ←((SP)), (SP) ←(SP)+2

注: IRET 指令执行完, 标志位由堆栈中取出的值确定

（六）CPU 控制指令

（1）标志位处理指令

CLC CF=0; STC CF=1; CMC CF 求反;

CLD DF=0; STD DF=1;

CLI IF=0; STI IF=1;

（2）同步控制指令

WAIT 等待指令, 通常用在 ESC 之后, 等待 TEST* 的有效信号。当 TEST*=1, 继续执行 WAIT, 每隔 5 个时钟周期测试一次;

若 TEST*=0, 结束 WAIT 指令。

ESC ext_op, s 交换指令, 使某个协处理器可以从 CPU 程序中获取一条指令或者一个存储器操作数

LOCK 总线封锁前缀

（3）其它

NOP 空操作指令, 占用 3 个时钟周期, 用作延时

HLT 暂停指令, 使 CPU 挂起至收到复位或中断信号

4. 2 汇编语言基本语法

1. 语言的种类:指令语句 (执行性), 伪指令语句 (说明性) 和宏指令语句 (定义) **注: 区别: 指令必须生成机器代码, 然后在程序运行期间由 CPU 来执行其操作; 伪指令是在汇编期间由汇编程序执行的操作命令, 本身并不生成目标代码**

2. 指令语句

格式: [标号:] [前缀] 指令助记符 [操作数表] [; 注释]

(1) 标号: 表示指令地址, 是指令的符号地址, 它具有 3 种属性——段基址、段内偏移量以及类型 (NEAR/FAR), 一般只在循环、转移和调用指令中使用

(2) 操作数: 立即操作数, 寄存器操作数, 存储器操作数

注: 当基址寄存器为 BP 时, 相对应的段寄存器为 SS; 可以采用“段超越”前缀, 来改变段寄存器, 串操作有特例

(3) 表达式:

A. 常量: 包括数值常量和符号常量 (= / EQU)

B. 数值表达式: 算术运算符: +、-、*、/、MOD、SHR、SHL

逻辑运算符: AND、OR、XOR、NOT

关系运算符: EQ(=), NE(≠), LT(<), GT(>),

LE(≤), GE(≥), 当关系成立的时候, 结果为 FFFFH, 否则为 0 例: AND AX, ((NUMB LT 5) AND 30) OR ((NUMB GE 5) AND 20)

操作符 AND 与操作数表达式中的 AND 具有不同的含意, 前者是指令助记符, 后者是伪运算

C. 变量: “变量”是操作数的符号地址, 汇编程序将操作数的第一个字节的偏移地址赋予这个变量

类型: DB 1 字节 / DW 2 字节 / DD 4 字节 / DQ 8 字节 / DT 10 字节

注: 1. 指令中必须明确是完成 8 位数据操作还是 16 位;

2. 变量作为寄存器操作数使用时, 段属性必须和该指令默认的段寄存器内容相符, 否则要使用段超越;

3. “变量”和“标号”的区别:

变量指向的是数据段或者者附加段中的某个数据项, 标号指向的是代码段某条指令; 变量的类型是数据项存取单位的大小, 标号的类型是 NEAR 或 FAR。

D. 地址表达式: 表示存储器地址, 其值一般为段内偏移地址

1) 加超越运算符: +、-

2) [] : 表示存储器地址

3) PTR: 用于说明变量、标号以及地址表达式的类型, 包括 (BYTE / WORD / DWORD / NEAR / FAR)

4) 段超越运算符: 如 MOV AX, ES: [23H]

注: 1. 变量或标号加或减某个结果为整数的数值表达式, 结果仍为变量或标号, 指向某存储单元; 例: MOV AL, DATA+5 和 MOV AL, [DATA+5] 两句结果相同;

2. 同一段内的变量或者标号可以相减, 其结果不是地址, 而是一个数值, 该数值表示两者间相距的字节数;

3. 方括号及寄存器 BX BP SI DI, 如这几个寄存器不用方括号括起来, 表示寄存器本身或操作数, 否则表示地址表达式。

E. 运算符综述 (算术 逻辑 关系 分析 合成)

1) 分析运算符

SEG——变量或标号的段地址

OFFSET——回送偏移值

TYPE——回送反映变量或标号类型的一个数值, 若是变量, 则数值为字节数, 如 DB 为 1, DW 为 2; 若是标号, 则数值为表示标号类型的数值, 即 NEAR 为 -1, FAR 为 -2

SIZE——回送变量数据区的数据字节总数

LENGTH——回送变量数据区的数据项总数

HIGH——地址表达式或 16 位绝对值的高 8 位

LOW——地址表达式或 16 位绝对值的低 8 位

2) 合成运算符 (均和 EQU 联合使用)

PTR: 建立一个新的变量或标号, 段地址和偏移量由右边操作数决定, 类型由左边决定, 例: ONE BYTE EQU BYTE PTR TWO BYTE

THIS: 建立一个指定类型的存储器地址或者操作数, 段地址和偏移量均与下一个能分配的存储单元相同。

3. 伪指令语句

格式: [名字] 伪指令助记符 [参数表] [; 注释]

(1) 数据定义伪指令

[变量] 助记符 操作数[, ..., 操作数] [; 注释]

助记符: DB DW DD DQ DT

1) “变量”是操作数的符号地址, 可有可无, 作用与指令语句前的标号相同, **区别是变量后面不加冒号**。如果语句中有变量, 汇编程序将操作数的第一个字节的偏移地址赋予这个变量。

2) 操作数的形式可以是常量, 数值表达式, 地址表达式, ‘?’’, n DUP (?. 表达式)

DB 是唯一能定义字符串的伪操作, 字符串用 “ ” 括起来, 串中的每个字符占用一个字节, 在存储器内用相应的 ASCII 码表示。

4) \$ 用在伪指令的参数字段时, 它所表示的是地址计数器的当前值

5) 操作数为地址表达式: 数据定义时, 出现在地址表达式中的变量表示该变量的偏移地址, 用根据偏移地址计算出的地址表达式的值初始化相应的存储单元

格式 1: DW 地址表达式

功能: 利用该地址表达式的计算值初始化相应的存储字

格式 2: DD 地址表达式

功能: 地址表达式计算结果对应的段地址 (高位字) 和偏移地址 (低位字) 来初始化相应的两个存储字。

(2) 符号定义伪指令

作用: 通过伪指令对符号重新命名或定义新的类型属性

A. 名字 EQU 表达式

1) 为常量定义符号; 例: ONE EQU 1

2) 为变量或标号定义新的类型属性并重命名; 例: 同 PTR

3) 为地址表达式所指的任意存储单元命名 XYZ EQU [BP+3]

4) 为符号定义新名字; 例: COUNT EQU CX

注: EQU 伪操作不允许重复定义

B. 名字 = 表达式

注: 与 EQU 基本相同, 区别在于: = 伪操作允许重复定义

习惯上, 一般用于定义符号常量

C. 变量名/标号名 LABEL 类型

1) LABEL 可以使指向同一位置的不同变量或标号具有不同的类型属性, 类型可以是 BYTE、WORD、DWORD、NEAR、FAR

2) 不仅给名字 (标号或变量) 定义, 还隐含由给名字定义段属性和偏移量属性。**注: 同 THIS 相似, 由接下来定义的存储单元决定。**

例: SUBF LABEL FAR; SUBN: SUB AX, [BX+SI+3] 标号 SUBF 和 SUBN 指向代码段的同一地址, 具有相同的段值属性和偏移值属性, 只是类型不同

(3) 段定义伪指令

A. 段名 SEGMENT [定位类型] [组合类型] [‘类别’]

· · · · (段体)

段名 ENDS

1) 定位类型: 表示该段对起始边界地址的要求:

BYTE 起始地址任意, 即字节型;

WORD 起始地址为偶数, 即字;

PARA 起始地址 XXXX0H, 即字节 (16BYTE 为 1 节);

PAGE 起始地址 XXX00H, 即页面 (256BYTE 为 1 页)。

注: 其中 PARA 为缺省值, 即如果省略“定位类型”, 则汇编程序按 PARA 处理

2) 组合类型: AT 表达式可以用于指定段地址, 段地址 = 表达式的值, 其值必为 16 位. 但 AT 不能用来指定代码段;

NONE: 表示此段与其他段无关联 (为缺省项);

PUBLIC: 表示此段与其他段用 PUBLIC 说明的同类别的段连接成一个逻辑, 装入同一物理地址中;

STACK: 将具有此说明类型的同名段连接成一个大的堆栈, 运行时, SS 和 SP 指向堆栈段的开始位置;

COMMON: 与其他此类型的段重叠地放在一起, 长度由最长的决定, 使不同模块的变量和标号共用一存储区域;

MEMPORY: 放在所有段的最后 (高地址端), 若出现多个此说明, 则第一个为 MEMPORY, 其余为 COMMON。

B. ASSUME 伪指令格式: ASSUME 段寄存器名: 段名, ...

1) 其中段寄存器名必须是 CS、DS、ES 和 SS 中的一个;

2) 段名必须是由 SEGMENT 定义的段名, 或是表达式 “SEG 变量” 或 “SEG 标号”, 或是 NOTHING;

3) 由于 ASSUME 伪指令只是指定某个段分配给哪一个段寄存器, 它并不能把段地址装入段寄存器中, 所以在代码段中, 还必须把段地址装入相应的段寄存器中。在程序中不需要用指令装入代码段的段地址, 因为在程序初始化时, 装入程序已将代码段的段地址装入 CS 寄存器了;

4) 还可以用 NOTHING 来取消这个对应关系。

C. ORG 伪指令

用来表示起始的偏移地址, 紧接着 ORG 的数值就是偏移地址的起始值。ORG 伪操作常用在数据段指定数据的存储地址, 有时也用来指定代码段的起始地址。

例: DATA SEGMENT

ORG 0010H ; 指定偏移地址为 0010H

(4) 过程定义伪指令

过程名 PROC [类型] / / 过程名 ENDP

过程名: 标识符, 是子程序入口的符号地址, 与标号作用相同

A. 类型:

1) 如果调用程序和过程在同一个代码段中, 使用 NEAR 属性。

2) 如果调用程序和过程不在同一个代码段中, 使用 FAR 属性。

B. 分类:

1) 外部过程: 调用该过程的主程序与该过程不在同一源文件中。此时需要在主程序文件中说明该过程 (设过程名为 PROC0):

EXTRN PROC0: FAR

同时在定义该过程的文件中说明该过程可以被其它程序文件调用: PUBLIC PROC0

2) 内部过程: 调用该过程的主程序和该过程在同一个源文件中。内部过程又分为段内过程和段外过程。过程的调用和返回:

C. CALL 和 RET:

CALL 使返回地址入栈, RET 使返回地址出栈

对于远过程: CALL 下一条指令段地址和偏移地址入栈

SP ← SP-2, [SP+SS*16] ← CS;

[SP] ← SP-2, [SP+SS*16] ← IP;

对于近过程: CALL 下一条指令偏移地址入栈

SP ← SP-2, [SP+SS*16] ← IP;

过程返回: RET / RET n

4. 3 汇编语言程序设计

1. DOS 及 BIOS 中断调用

(1) 中断向量: 中断例程程序的入口地址, 存放于中断向量区。每个中断向量占 4 个字节, 共 256 个中断, 从 00000H~003FFH, 占用了 1024 个字节 (1K 单元), 可转到 1MB 空间的任何位置。

规定 INT 21H 为进入各功能调用子程序地总入口。

(2) 基本步骤:

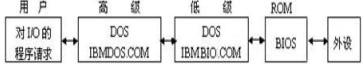
- 1. 将调用参数装入指定的寄存器中;
- 2. 如需功能号, 把它装入 AH;
- 3. 按中断号调用 DOS 或 BIOS 中断;
- 4. 检查返回参数是否正确。

注: 中断向量分配情况:

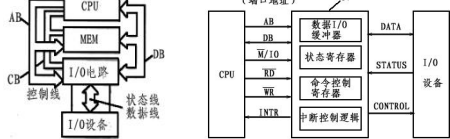
0~1FH, 80H~FOH 为 BIOS 中断向量号; 20H~3FH 为 DOS 中断向量号; 40H~7FH 为用户备用

(3) 常用 DOS 中断调用: (功能号总结)

- 4CH——返回 DOS
- 1——键盘输入并显示 (存放在 AL)
- 8——键盘输入但不显示
- 2——显示一字符 (DL=显示字符的 ASCII 码)
- 5——打印一字符 (DL 同上)
- 9——显示以\$结尾的字符串 (DS:DX 指向字符串首地址)
- 0AH——字符串输入 (DS:DX 指向缓冲区)
- 3——异步通信口输入; 4——异步通信口输出
- 2BH——设置日期 (CX DH DL 代表年 1980~2099 月 日)
- 2AH——取日期
- 2DH——设置时间 (CH CL DH 代表时 分 秒; DL 为百分之一秒)
- 2CH——取时间



6.1 输入输出与中断概述



1. 接口电路传送的信息可分为 3 类:

数据信息: 数字量 (二进制或 ASCII) / 模拟量 / 开关量 (0 或 1)

状态信息: Ready/Busy

控制信息: 控制外设的启动/停止

作用: 电平转换、数据格式转换、协调速度、总线隔离

- 2. 3 种信息的性质不同, 通过不同的端口分别传送
- 3. 在 8086 中, 外设的状态为输入数据, CPU 的控制命令为输出数据, 通过数据总线分别传送
- 4. 端口地址由 CPU 地址总线的低 8 位或低 16 位地址信息确定

6.2 CPU 与外设之间的数据传送方式

1. 程序传送方式

通过 IN 和 OUT 指令直接控制 I/O 和 CPU 之间数据交换的方式

无条件传送 (同步传送)

简单快捷的外部设备采用同步方式, I/O 接口可以是简单的数据锁存缓冲器, CPU 如同访问内存存储器一样访问这些外设接口, 无需检测其状态。

条件传送 (异步传送) (程序查询传送)

在执行输入 IN 或输出 OUT 前, 要先查询接口中状态寄存器的状态。输入时, 该状态信息指示要输入的数据是否已“准备就绪” (READY); 输出时, 指示输出设备是否“空闲” (BUSY), 由此条件来决定执行输入或输出。

(1) 程序查询输入:

若 READY=0, CPU 继续等待查询, 至 READY=1, 执行下一步; 执行输入指令后, 是状态信息清零, 即 D 触发器复位。

(2) 程序查询输出:

若 BUSY=1, CPU 继续等待查询, 至 BUSY=0, 执行下一步; 执行输出指令后, D 触发器置 1, 一方面通知外设可以执行输出, 另一方面告知 CPU 外设 BUSY, 组织 CPU 输出新数据。

(3) 执行输入/输出指令, 进行 I/O 传送。完成数据输入/输出, 同时将外设的状态信息复位, 一个 8 位数据传送结束。

注: 缺点: CPU 必须作程序等待循环, 不断测试外设的状态, 直至外设为交换数据准备就绪时为止, 大大降低了 CPU 的运行效率。

2. 中断传送方式

利用中断来实现 CPU 与外设之间的数据传送方式

从外设启动到准备就绪, 查询方式处于等待状态, 而中断传送仅在外设准备好数据传送的情况下才中止 CPU 执行的主程序, 一定程度上实现了主机和外设并行工作, 提高了 CPU 的利用率。

3. 直接存储器存取方式 (DMA 方式) (成组数据传送方式)

通过 DMA 控制器, 使 I/O 设备直接与内存高速交换数据

DMA 控制器

四个寄存器: 信息传送前由系统程序进行初始化

控制寄存器: 设置控制字, 指出数据是输入还是输出, 并启动状态寄存器; DMA 操作;

地址寄存器: 设置要传送的数据块的首地址;

字节计数器: 设置要传送的数据长度 (字节数);

每个字节传送后地址寄存器增 1, 字节计数器减 1。

步骤:

- (1) 一个 DMA 控制器可以连接多个 I/O 设备, I/O 设备向 DMA 控制器发出 DMA 请求;
- (2) DMA 控制器向 CPU 发出总线请求 (HOLD 变为高电平);
- (3) CPU 完成当前状态操作后, (HLDA 输出高电平) 响应 DMA 控制器的请求。同时 CPU 交出总线的控制权给 DMA 控制器。此时, DMA 控制器成为系统的总线主设备;
- (4) DMA 通知 I/O 外设, 开始 DMA 数据传输;
- (5) DMA 传送结束, HOLD 信号变低电平, 撤销总线请求;
- (6) HLDA 信号变低电平, CPU 恢复对总线的控制权。

注: 三种数据传送方式的比较:

程序控制方式: 对于条件传送, CPU 需反复查询外设状态, 当外设已准备好数据或不忙碌时, 才能与 CPU 交换数据, CPU 的利用率很低。

中断方式: 只有输入设备将数据准备好或输出锁存器为空时才能向 CPU 发中断请求。CPU 响应后将暂停执行的主程序, 转而执行中断服务程序, 使 CPU 和外设交换一次数据, 再返回主程

序。由于不需要反复查询外设状态, 因此 CPU 利用率大大提高。DMA 方式: 由 DMA 控制器取代 CPU 接管总线 (传送数据不经 CPU 中转), 控制外设与内存间通过总线直接进行高速成批的数据交换, 传输效率高, 数据传输速度基本取决于外设和存储器的存取速度。

6.3 8086/8088 中断系统

1. 基本概念

中断源: 中断申请的来源

外部设备、实时时钟、故障源属于随机中断源, 引起强迫中断; 调试程序设置的中断源 (断点、单步调试等), 引起自愿中断; 外部 (硬件) 中断: NMI 不可屏蔽中断 (INT 2) (主板上 RAM 的奇偶校验错; 扩展槽中的 I/O 通道错;

浮点运算协处理器 8087 的中断请求)

INTR 可屏蔽中断 (取决于 IF 状态)

内部 (软件) 中断: INT n 用户定义软中断

INT 3 断点中断 (000CH~000FH)

INT 1 单步中断 (TF=1) (0004H~0008H)

INT 0 除法错误 (0000H~0003H)

INT0 (INT 4) 溢出中断 (0010H~0013H)

注: 内部中断的特点:

- (1) 由一条 INT n 指令直接产生;
- (2) 除单步中断以外, 都不能被屏蔽;
- (3) 因不必通过查询外设来获得中断类型号, 故没有响应 INTA* 的总线周期;
- (4) 除单步中断, 所有软件中断优先级都高于外部中断;
- (5) 使用断点中断 (INT 3) 逐段调试程序时, 可用中断服务程序在屏幕上显示各种信息;
- (6) INT nn 可模拟外设提供的硬件中断 (避免产生 INTR 信号和提供中断类型号), 使 nn 类型号与外设类型号相同即可。

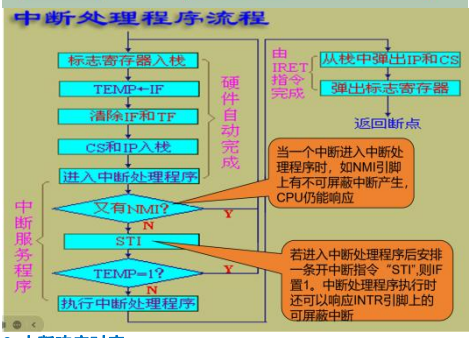
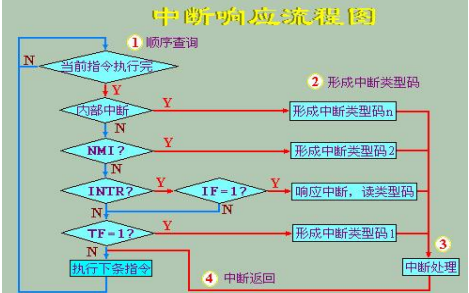
中断优先级: INT 0 除法错误 > INT n > INTO 溢出中断 > NMI 不可屏蔽 > INTR 可屏蔽 > INT 1 单步中断

中断嵌套: 高级中断源 (优先级高) 能够中断低级中断源的中断服务程序, 转而执行高级中断服务程序, 执行完成后, 再返回执行低级中断处理程序, 最后返回主程序, 形成嵌套结构。

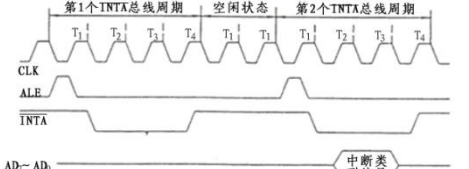
中断向量表: 存放中断服务程序的入口地址。 (共 256 个中断)

2. 中断处理过程

中断请求→中断响应→中断处理→中断返回



3. 中断响应时序



由两个连续的 INTA* 中断响应总线周期组成: 第 1 个表示 CPU 通知外设中断请求已获允许; 第 2 个, 外设通过 16 位数据总线的低 8 位将中断类型号发送给 CPU。中间由两个空闲时钟周期 T1 隔开 (INTA* 输出为低电平)。

4. 可屏蔽中断的具体过程 (中断处理过程+响应时序举例)

- (1) 外设产生中断请求信号由 INTR 送至 CPU
- (2) 若 IF=1 (开中断), 且 CPU 完成当前指令后, 便响应中断, 通过 INTA* 向中断接口电路发出响应信号
- (3) CPU 执行第一个 INTA 周期: 通知外设接受请求
- (4) CPU 执行第二个 INTA 周期: 获得中断类型号
- (5) 总线写周期: 写入 PSW (状态字) (标志寄存器)
- (6) 总线写周期: CS 入栈
- (7) 总线写周期: IP 入栈
- (8) 清除 IF 和 TF
- (9) 总线读周期: 读入 CS
- (10) 总线读周期: 读入 IP
- (11) 转入执行中断服务程序
- (12) (一般情况下) STI 开中断指令, 允许中断嵌套
- (13) IRET 中断返回指令, 恢复现场

总结分析:

A. 总线周期: 在执行中断响应程序前 CPU 实际执行 7 个周期;

B. 非屏蔽 NMI 与软件中断不需要 (1) ~ (4);

C. 在 (1) 中, 中断源向 CPU 发信号的条件: 设置中断请求触发器; 设置中断屏蔽触发器;

D. 在 (2) 中, CPU 响应中断的条件: CPU 开放中断 (IF=1); CPU 现行指令结束;

E. CPU 响应中断的过程及处理:

关中断: CPU 响应后, 内部自动 (由硬件) 实现, 避免在处理当前中断时又被新的中断打断, 以致破坏现场

保留断点 (CS 和 IP);

保护现场 PUSH (寄存器内容和标志位状态);

恢复现场 POP (寄存器内容和标志位状态);

开中断: STI 指令, 允许中断嵌套。一般在转入中断服务程序后; 返回: IRET (CS 和 IP)。

5. 中断服务子程序设计

(1) 选择一个中断向量号。40H~7FH 中断向量是留给用户增加中断服务程序时使用的;

(2) 将中断子程序的入口地址置入中断向量表的相应表项中;

注: 其置入方法有两种:

A. 用数据传送指令

B. 采用 DOS 重新中断向量的中断功能:

向量号 21H
功能号 25H
入口参数: DS=中断服务子程序入口段地址;
DX=中断服务子程序入口相对地址;
AL=新增的向量号

(3) 使中断服务子程序驻留内存, 之后其占用的存储区就不会被别的软件覆盖。

注: 程序驻留内存的方法是采用 DOS 的中断调用:

向量号 21H
功能号 31H
入口参数: DX=驻留程序字节数

例: 在微机中增加一中断服务子程序, 其向量号为 50H, 其功能是 BX 内容增 1。

```
C SEGMENT
ASSUME CS:C
ORG 100H ;设定程序的入口地址为100H
B: MOV AX,SEG SUBP ;中断服务子程序入口段地址写入DS
MOV DS,AX ;中断服务子程序入口偏移(相对)地址写入DX
MOV DX,OFFSET SUBP ;新增的中断向量号写入AL
MOV AL,50H
MOV AH,25H
INT 21H ;将段地址、偏移地址置入中断向量表相应表项
MOV DX,N
MOV AH,31H
INT 21H ;使中断服务子程序驻留内存并返回DOS
SUBP PROC FAR
INC BX
IRET
SUBP ENDP
N EQU $-SUBP
C ENDS
END B
```

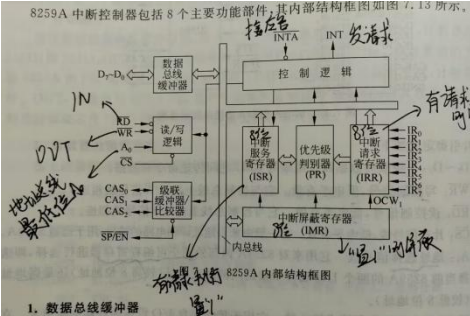
7.3 可编程中断控制器 8259A

1. 结构和工作原理

引脚与功能结构:

- (1) DT/D0: 双向数据线, 用于与 CPU 传送命令和数据。
- (2) WR*: 写控制, 低电平有效, 与控制总线上的 10W* 相连。
- (3) RD*: 读控制, 低电平有效, 与控制总线上的 10R* 相连。
- (4) CS*: 片选信号线, 低电平有效, 用于选通 8259A。
- (5) A0: 地址信号选择线, 用于对两个可编程寄存器进行选择 (即两个 I/O 端口被访问的是奇地址 (高 8 位) 还是偶地址 (低 8 位))。
- (6) IR0~IR7: 8 级中断请求输入线, 用于接收外部中断请求。
- (7) INT: 中断请求信号 (输出), 与 CPU 的 INTR 相连。
- (8) INTA*: 中断响应信号线 (输入), 与 CPU 的 INTA* 相连。
- (9) SP*/EN*: 当 8259A 采用缓冲连接总线方式时, 为 EN* (输出), 有效时允许数据总线缓冲器选通, 数据由 8259A 读出至 CPU, 无效表示数据由 CPU 写入 8259A; 当采用非缓冲方式 (主从工作方式) 时, 为 SP* (输入), 区分主从, SP*=1 为主片, SP*=0 为从片。
- (10) CAS0~CAS2: 3 级级联控制信号, 主从结构中, 最多扩展 8~64 级主从式中断请求, 主片为输出信号, 从片为输入信号。

内部结构:



(1) 数据总线缓冲器

双向的 8 位三态缓冲器, 用作 CPU 与 8259A 之间的数据接口。

(2) 读/写逻辑

接收 CPU 的读/写控制信号 (RD*/WR*) 和片选控制信号 (CS*), 及一位地址信号 (A0)。CPU 执行 IN 指令时, RD* 与 A0 配合; 执行 OUT 指令时, WR* 与 A0 配合, 控制命令字 (CPU) 写入某个指定的内部寄存器。注: 将 CPU 地址总线的最低位 A0 接到 8259A 的 A 端即可选定某个端口。

(3) 级联缓冲器/比较器

提供级联控制与双向功能信号, 满足缓冲与非缓冲工作的需要。

(4) 控制逻辑

内部控制电路, 用于向 CPU 发中断请求信号 (INT) 或接收来自 CPU 的中断应答信号 (INTA)。

(5) 中断请求寄存器 IRR (8 位寄存器)
当某个 IRI 接收到高电平的中断请求信号时, 相应位 IRRi 将被置 1; 最多 8 个信号同时进入, IRR 被置为全 1; 能否进入 PR 取决于 IRR 相应位是否清 0。

(6) 中断服务寄存器 ISR (8 位寄存器)
存放或记录正在服务中的所有中断请求。当某级中断请求被响应 (CPU 执行中断), ISR 中的相应位置 1, 并保持到该级中断处理过程结束为止。多重中断时, ISR 可能有多位同时被置 1。
过程: 一个或多个中断源同时请求中断→PR 选出 IRR 中置 1 的各种中断优先级别最高者→INTA*负脉冲选通送入 ISR 对应位。

(7) 中断屏蔽寄存器 IMR (8 位寄存器)
屏蔽已被锁存于 IRR 中的任何一个中断请求级。相应位置 1 即可屏蔽对应 IRR 位置的中断请求线。使之不能进入 PR 去判优。

(8) 优先级判别器 PR
判定中断请求最高优先级, 若当前没有正在执行的中断, 则系统首先响应并执行这一级中断; 若有, 则 PR 判定是否允许中断嵌套 (接收并比较 ISR 和 IRR 中的中断优先级), 若允许, 则 INT=1, 向 CPU 发出新的中断请求。

2. 中断工作过程
单片方式中断工作流程
①外部中断源出现中断请求→IR 高电平→IRR 相应位置 1;
②若 IRR 对应的 IMR 位为 0→IRR 有效位送 PR→并和 ISR 比较;
③若 IRR 优先级高→INT 有效→中断请求信号送 CPU INTR 引脚;
④若 CPU 的 IF=1 (开中断)→执行当前指令→发送 INTA*1 置 8259A 相应的 ISR 为 1, 清除相应的 IRR→发送 INTA*2, 8259A 将对应的中断类型号 (8bits) 从中断类型寄存器通过数据线送至 CPU, CPU 从中断向量表中中断入口, 进入中断服务程序;
⑤结束中断操作: AE01 (自动结束中断) 模式 (仅用于单片), INTA*2 结束时使 ISR=0; E01 模式, 则保持到中断服务程序结束时由 CPU 发 E01 命令清零 ISR。

级联方式中断工作流程
级联: 用多片 8259A 构成主从关系, 级联使用。最多 8 个从片, 可以扩展为 64 级中断。CASO~CAS2 相当于地址线, 主从片的 CASO~CAS2 互连接, 主片根据发出中断请求的从片与 IR 连接的情况判断是哪一级从片, 再通过 CASO~CAS2 发出选择信号。
①某 Slave IRI (s) 有效, 相应的 Master IRj (m) 有效, 经优先判别后, 主片向 CPU 发出中断请求;
②CPU 允许中断请求后发出 INTA*1, 8259 (m) CAS2~CAS0 有效, 应答对应的 8259 (s);
8259 (s): ISRi=1, IRRi=0
8259 (m): ISRj=1, IRRj=0

等同于单片方式中置 8259A 相应的 ISR 为 1, 清除相应的 IRR
③发出 INTA*2, 此时 CAS2~CAS0 仍有效, 8259 (s) 送类型号;
④CPU 从中断向量表中取中断入口地址, 进入中断服务程序;
⑤中断结束, CPU 分别对主、从片发两次 E01, 使各自 ISR=0;
注: 级联中主从 8259A 分别初始化, 结束时分别发送对应的 E01。

3. 命令字
8259A 为可编程中断控制器, 可以通过程序对 8259A 写入命令字, 设置 8259A 工作方式以及发出各种命令。
初始化命令字: ICW1~ICW4
对 8259A 进行初始化, 要求按一定顺序写入。在系统运行过程中一般不再改变。



(2) ICW2: 中断类型号, 8 位中断类型寄存器

A0	D7	D6	D5	D4	D3	D2	D1	D0
1	T7	T6	T5	T4	T3	0	0	0

标记: A0=1;
无关项: D2~D0 中断类型号的数值与低三位无关, 可以任选;
D7~D3: 用于确定 8 个连续向量中断的起始类型号, 表示中断类型号的高五位, 低 3 位由中断请求引脚 IRO~IR7 确定。例: 若 8259A 的中断向量为 70H~77H, 则将 01110 装入命令字 ICW2。
(3) ICW3: 标志主/从片, 8 位主/从寄存器 (只用于级联方式)
主 8259A (输入端 SP*=1) 从 8259A (输入端 SP*=0)

(4) ICW4: 中断结束方式, 8 位方式控制寄存器

A0	D7	D6	D5	D4	D3	D2	D1	D0
1	0	0	0	SFNM	BUF	M/S	AE01	uPM

标记: A0=1, D7~D5=000 (识别码) (ICW1 中的 D0=1);
D4: SFNM=1, 为特殊的全嵌套方式; SFNM=0, 为 (一般) 非特殊的全嵌套方式;
D3: BUF=1, 缓冲方式; BUF=0, 非缓冲方式;
D2: M/S*=1, 主 8259A; M/S*=0, 从 8259A;

0X 非缓冲方式;
BUF、M/S=10 缓冲方式 (从片 8259A 作从片);
11 缓冲方式 (从片 8259A 作主片)。

D1: AE01=1, 采用自动结束中断方式, 否则为非自动结束方式;
D0: uPM=1 表示 8259A 当前处于 8086/8088 系统, uPM=0 表示处于 8080/8085 系统。

操作命令字: OCW1~OCW3
初始化后的任何时刻都可写入, 对中断过程进行动态的操作与控制, 操作过程中允许重新设置。

(1) OCW1: 写入 IMR 寄存器, 用于屏蔽/允许中断请求

A0	D7	D6	D5	D4	D3	D2	D1	D0
1	M7	M6	M5	M4	M3	M2	M1	M0

标记: A0=1;
Mi=1→IMRi=1→IRi 的中断请求被屏蔽
Mi=0→IMRi=0→IRi 的中断请求被允许

(2) OCW2: 设置优先级循环方式 + 中断结束方式

A0	D7	D6	D5	D4	D3	D2	D1	D0
0	2	2	2	2	2	2	2	2

标记: A0=0, D3=D4=0;
L2~L0: 对于特殊中断结束命令 (SE01), 指出要清除的 ISRi; 对优先级特殊循环方式命令, 指出循环开始时最低的中断优先级 (000~111 对应 IRO~IR7);
D7: R=1, 优先级自动循环方式; R=0, 优先级非自动循环方式;
D6: SL=1, 特殊中断结束, L2~L0 有效, SL=0, 一般中断结束
D5: E01=1, 使当前 ISR 中的对应位复位;
R、SL E01: (功能说明)

001—E01 一般结束方式+优先级非循环 (一般用于全嵌套)
011—SE01 特殊结束方式 (一般用于非全嵌套)
101—E01+优先级自动循环 (将当前正在处理的中断服务对应的 ISRi 清 0, 并将对应的 IRI 降为最低优先级, 最高优先级赋予 IRI+1)
111—SE01+优先级自动循环
000—AE01+清除优先级自动循环
100—AE01+优先级自动循环
110—优先级特殊循环方式 (初始最低优先级看 L2~L0)
010—OCW2 无意义

(3) OCW3: 控制中断屏蔽, 设置查询方式, 读内部寄存器

A0	D7	D6	D5	D4	D3	D2	D1	D0
0	0	ESMM	SMM	0	1	P	RR	RIS

D1 D2 操作
1 0 读 IRR
1 1 读 ISR
1 0 恢复原先优先级的工作方式
1 1 特殊屏蔽工作方式, 优先级不起作用, 若 IF=1, 则可以响应任何一级未屏蔽的中断请求

标记: A0=0, D7=D4=0, D3=1;
D6: ESMM 为特殊屏蔽模式允许位 D5: SMM 为特殊屏蔽模式位
D1: RR 为读寄存器位, 表示是否允许读 8259A 的状态
D0: RIS 为读 ISR/IRR 选择位
D2: P 为查询方式位, P=1 时, 把下一个读脉冲 (RD*信号) 作为中断响应 INTA*信号送 8259A, 可以得知中断请求信息

4. 8259A 的基本工作方式
缓冲/非缓冲方式 (由命令字 ICW4 中的 D3 位设置)
①缓冲方式: 级联系统中, 8259A 需经驱动器才能与总线相连, 此时 SP*/EN*起驱动作用。
②非缓冲方式: 单片或少量 8259A 系统, 8259A 可直接与数据总线相连, 此时 SP*/EN*用于表示主从。
8259A 优先级的设置方式
A. 特殊全嵌套/一般全嵌套方式 (由 ICW4 中的 D4 位设置)
① (一般) 非特殊全嵌套方式: 中断 0 级最高, 7 级最低, 只能嵌套高级请求, 不响应同级请求。 (缺省)
注: (一般) 全嵌套方式是系统默认缺省优先级工作方式
②特殊全嵌套方式: 优先级同上, 允许响应或嵌套同级的中断请求。同一从片来的不同级别的中断请求, 对于主片来说是同一级的, 但对于从片来说, 新的中断请求必须比正在服务的中断优先级高, 否则, 从片就不会发出 INT 信号。
B. 优先级自动循环/特殊循环方式 (由 OCW2 设置)
③优先级自动循环方式: 希望多个中断源被服务的机会均等, 多个中断优先级轮流优先, 一个中断请求设备被服务以后, 它的优先级自动降为最低。
④优先级特殊循环方式: 与优先级自动循环方式相似, 不同在于初始状态的最低优先级由程序确定。

中断结束的三种方式 (由 ICW4 中的 D1 位以及 OCW2 控制)
①AE01 自动结束方式 (仅用于单片): INTA*2 结束时 ISR=0。
注: 在 AE01 模式中, INTA*2 结束后的中断服务程序过程, 若有同级的或低级的中断请求, 都会被允许而发生中断嵌套。因此, 这种模式仅用于单片 8259A 且不可能产生中断嵌套的情况。
②E01 一般结束方式 (用于全嵌套): ISR 保持到中断服务程序结束时由 CPU 发 E01 命令给 8259A 清零 ISR。
注: 在全嵌套方式中, 若在 ISR 中有两个以上的位被置 1, E01 命令将清除 ISR 中优先级最高的有效 (置 1) 位。
③SE01 特殊结束方式 (用于非全嵌套): 由 CPU 发 SE01 命令给 8259A 清零指定的 ISR。注: 非全嵌套方式下, 若正在服务寄存器中有两位以上被置 1, 控制器无法根据 ISR 判断哪一级中断是当前正在处理的中断, 故采用特殊的中断结束方式。

5. 寻址方式
8259A 的内部 7 个控制字寄存器寻址方法:
ICW1: A0=0, D4=1 偶地址写入, 且 D4=1
ICW2: A0=1
ICW3: A0=1
ICW4: A0=1
OCW1: A0=1
OCW2: A0=0, D3=D4=0
OCW3: A0=0, D7=D4=0, D3=1
采用了专门的“标识位” (D4D3) 节省了输入地址的引脚数 (A0)

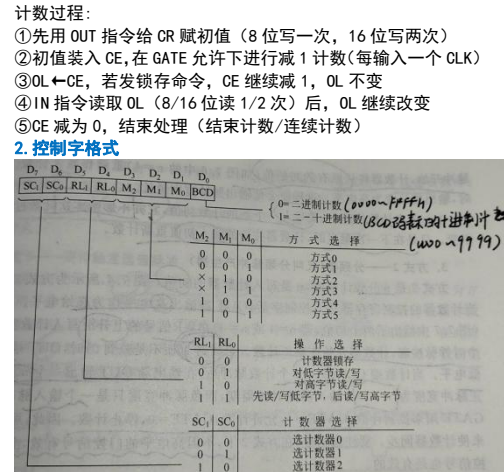
8259A 内部寄存器读操作寻址方法:
A0=0 时, 通过 OCW3 可读出 IRR 和 ISR; A0=1 时, 可读出 IMR

6. 编程应用
初始化 (例: 8259A 片选地址为 20H、21H)
MOV AL, 13H ; 写 ICW1, 单片, 边沿触发, 要 ICW4
OUT 20H, AL ; 若端口地址>255H, 要用 DX 来传送
注: MOV DX, 300H OUT DX, AL
MOV AL, 8 ; 写 ICW2, 中断类型号从 8 开始
OUT 21H, AL
MOV AL, 0DH ; 写 ICW4, 缓冲工作, 8086/8088 配置
OUT 21H, AL
MOV AL, 0 ; 写 OCW1, 允许 IRO~IR7 全部中断请求
OUT 21H, AL
送中断向量入口地址: 例: IR4 类型为 8+4=12 (0CH), 则入口地址为 (12*4=48) 30H~33H, 其中 30H~31H 存 IP
中断子程序结束
MOV AL, 20H ; 写 OCW2 命令, 是 ISR 相应位复位 (发送 E01)
OUT 20H, AL ; 若端口地址>255H, 要用 DX
IRET ; 开放中断允许并返回

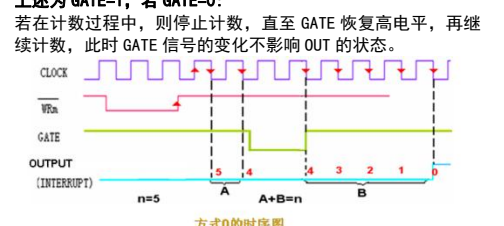
7.1 接口的分类及功能
1. 接口分类
功能: (1) 通用接口 (适用于大部分外设)
A. 并行接口: 并行接口是按字节传送
B. 串行接口: 和 CPU 之间并行传送, 和外设之间是串行传送
(2) 专用接口仅用于某台外设或微处理器, 增强 CPU 功能
灵活性: 硬布线逻辑接口芯片和可编程接口芯片
2. 接口功能
缓冲锁存数据; 地址译码; 传送命令; 码制转换; 电平转换

7.2 可编程计数器/定时器 8253-5 (最高计数频率为 2MHz)
1. 结构和工作原理
引脚及其功能:
(1) D0~D7: 数据线, 写控制字, 读写计数器数值;
(2) A1, A0: 地址线 (寻址方式)
00—计数器 0; 01—计数器 1;
10—计数器 2; 11—控制字寄存器;
(3) RD*: 读控制; (4) WR*: 写控制;
(5) CS*: 片选端 (=1 时无操作)
(6) CLK0~2: 计数器 0~2 时钟输入端
(7) GATE0~2: 计数器 0~2 的门控制脉冲输入端, 由外设送入
(8) OUT0~2: 计数器 0~2 的输出端

内部结构: 三个相同的 16 位计数器和一个 8 位控制字寄存器, 每个计数器内部包括计数初值寄存器 CR, 计数执行部件 CE 和输出锁存器 OL。
计数过程:
①先写 OUT 指令给 CR 赋初值 (8 位写一次, 16 位写两次)
②初值装入 CE, 在 GATE 允许下进行减 1 计数 (每输入一个 CLK)
③OL←CE, 若发锁存命令, CE 继续减 1, OL 不变
④IN 指令读取 OL (8/16 位读 1/2 次) 后, OL 继续改变
⑤CE 减为 0, 结束处理 (结束计数/连续计数)
2. 控制字格式

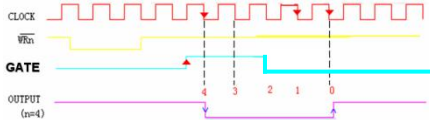


3. 工作方式
方式 0——计数产生中断
(1) 写入控制字, OUT 端输出低电平 (起始);
(2) 在有两个负脉冲宽度的 WR*信号的上沿沿 (将初值 n 写入 CR) 之后, 经一个 CLK 计数脉冲, 将 n 值装入计数器;
(3) 每经过一个 CLK 计数脉冲, 在 CLK 下降沿, 计数器减 1;
(4) 减到 n=0 时, 计数结束, OUT 由低变高 (可利用该电平变化向 CPU 发出中断请求), 并保持, 不开始重新计数。只有写入另一个计数值 m 时, 即刻触发发新值计数;
(5) 计数过程中可重新装入计数初值 m, 写完后, 计数器将从该值 m 开始重新计数。
上述为 GATE=1, 若 GATE=0:
若在计数过程中, 则停止计数, 直至 GATE 恢复高电平, 再继续计数, 此时 GATE 信号的变化不影响 OUT 的状态。

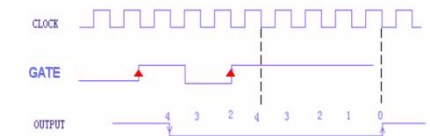


方式 1——可编程单稳态触发器
(1) 写入控制字, OUT 端输出高电平 (起始);
(2) 初值 n 送入计数器后, 必须等到 GATE 的上升沿出现, 才能在下一个 CLK 下降沿开始计数。计数开始时 OUT 由高变低, 每经过一个 CLK 计数脉冲, 计数器值减 1;
(3) 减至 n=0 时, 计数结束, OUT 由低变高, 并保持, 不开始重新计数。OUT 输出的单稳负脉冲的宽度=初值 n*tc (CLK 端输入脉冲周期);
(4) 若在计数过程中, GATE 由高变低, 不影响计数;

(5) 若在计数过程中, GATE 又输入一次上升沿, 则从 GATE 上
升沿之后的下一个 CLK 下降沿开始, 从 n 开始重新作减 1 计数
(可以改变脉冲的宽度)。



方式1的时序图 (计数过程中 GATE 仅有一个上升沿)



方式2的时序图 (计数过程中 GATE 不止产生一个上升沿)

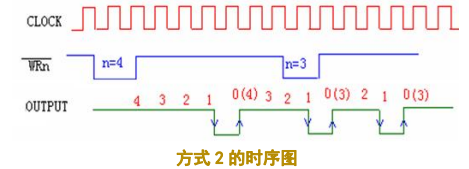
方式2——分频器

- (1) 写入控制字, OUT 端输出高电平 (起始);
- (2) WR*信号的上升沿将初值写入计数器, 之后从下一个 CLK 起, 每一个 CLK 下降沿, 计数值 n 减 1;
- (3) n 减至 1 时, OUT 由高变低; n 减为 0 时, OUT 又变高, 这时 CLK 正好输入了 n 个计数脉冲, OUT 相当于输出一个 n 分频脉冲, 正脉冲宽度为 (n-1), 负脉冲宽度为一个 CLK 宽度;
- (4) 接着自动装入 n, 并连续计数, 输出频率为 f (CLK) / n, 输出周期为 n*T (CLK);
- (5) 在计数过程中, 允许重新装入新的初值 m, 下一个计数周期按 m 值开始计数, 输出也会变成 m 分频脉冲。

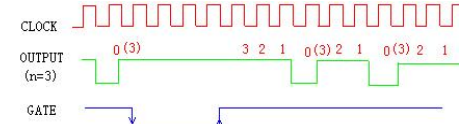
上述为 GATE=1, 若 GATE=0:

若在计数过程中, 则停止计数, 使 OUT 输出高电平, 直到 GATE 变为高电平后, 重新装入 n 值, 开始计数。

注: 在方式 2 下, GATE 的高电平和上升沿均有效。



方式2的时序图

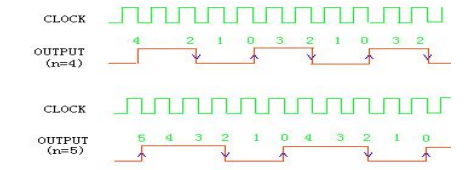


方式2的时序图 (GATE 电平改变)

方式3——方波频率发生器

- (1) 写入控制字后, OUT 端为低电平 (起始), 装入计数值 n 后, 又变为高电平;
- (2) n 为偶数, 每个 CLK 下降沿 n 值减 1, 至 n/2 时, OUT 由高变低, 并继续减 1 至 0, 相当于输出频率为 f (CLK) / n 的方波。当减至 n=0 时, OUT 再次变高, 重新装入 n, 继续计数;
- (3) n 为奇数, 输出高电平宽度为 (n+1)/2, 低电平宽度为 (n-1)/2 的方波, 其余同 n 为偶数时的情况;

上述为 GATE=1, 若 GATE=0: (GATE 功能同方式 2)

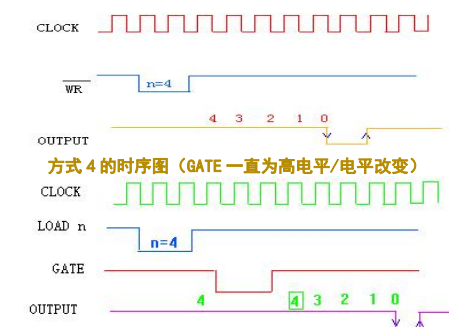


方式3的时序图

方式4——软件触发选通脉冲 (软件写入新的 n 来重新工作)

- (1) 写入控制字后, OUT 端为高电平 (初始), WR*信号上升沿写入初值 n 后, 每一个 CLK 下降沿, 计数值减 1, OUT 保持高电平不变。
- (2) 计数器减至 0 时, OUT 端输出一个 CLK 脉冲宽度的负脉冲, 然后停止计数, 只有输入新的计数值后, 才能开始新的计数。

上述为 GATE=1, 若 GATE=0: 若在计数过程中, 则停止计数, 直至其变高后, 重新将 n 装入, 开始计数。

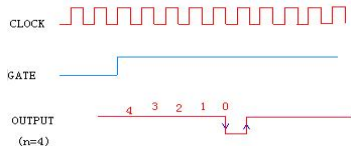


方式4的时序图 (GATE 一直为高电平/电平改变)

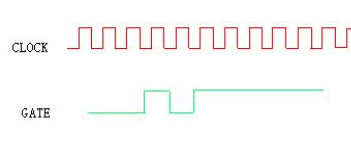
方式5——硬件触发选通脉冲 (硬件产生的 GATE 上升沿得到)

和方式 4 的 GATE 端输入信号功能不同:

- (1) GATE 的上升沿才可转入计数过程
- (2) 若计数过程中, GATE 由高变低, 不影响计数; 但 GATE 出现上升沿 (任何时刻) 将使得 n 重新装入计数器, 开始计数。
- (3) 减至 n=0 后, n 值会自动装入, 只要等待上升沿即可。



方式5的时序图 (GATE 不变/改变)



方式5的时序图 (GATE 改变)

4. 编程应用

使用 8253-5 时要进行初始化编程 (包括写入控制字和计数值)

- (1) 写控制字。计数器的选择由控制字的 D7D6 位来决定;
- (2) 写计数初始值。计数初始值由各计数器的端口地址写入。若控制字规定只写低 8 位, 则高 8 位自动置 0; 若规定只写高 8 位, 则低 8 位置 0。若为 16 位, 则分两次写入, 先低再高。

硬件结构: 三个计数器使用相同的时钟脉冲 CLK (1.19MHz), 端口地址为 40H, 41H, 42H; 控制寄存器的端口地址为 43H。

计数器 0: 电子时钟基准 (方式 3, 计数值预置为 0 (即 65536))

输出的方波频率为 1.19*10^6/65536=18.21Hz, 间隔为 55ms

程序: MOV AL, 36H ; 36H=00110110B
OUT 43H, AL ; 写控制字
MOV AL, 0 ; 预置计数值
OUT 40H, AL ; 先写低字节
OUT 40H, AL ; 再写高字节

计数器 1: 动态 RAM 定时刷新 (方式 2, 计数值预置为 18)

输出频率为 1.19*10^6/18=66.1kHz, 间隔为 15.1 μs

程序: MOV AL, 54H ; 54H=01010100B
OUT 43H, AL ; 写控制字
MOV AL, 12H ; 预置计数值
OUT 42H, AL ; 先写低字节
OUT 41H, AL ; 写低字节

计数器 2: 扬声器音调 (方式 3, 计数值预置为 1331 (即 533H))

输出频率为 1.19*10^6/1331=894Hz

程序: MOV AL, B6H ; B6H=10110110B
OUT 43H, AL ; 写控制字
MOV AL, 33H ; 预置计数值
OUT 42H, AL ; 先写低字节
MOV AL, 05H ; 预置计数值
OUT 42H, AL ; 再写高字节

7.3 可编程并行通信接口芯片 8255A

1. 结构与工作原理

引脚及其功能:

- (1) D0~D7: 数据线;
 - (2) A1, A0: 地址线 (寻址方式)
 - (3) RD*: 读控制;
 - (4) WR*: 写控制;
 - (5) CS*: 片选端;
- 注: 读写同时为 1 或 CS*=1 均为无操作; 控制字不可读。
- (6) RESET: 复位信号, 8255A 复位后, 所有 I/O 为输入状态;
 - (7) A 口: 8 位数据输入锁存器和 8 位数据输出锁存器/缓冲器;
 - (8) B/C 口: 8 位数据输入缓冲器和 8 位数据输出锁存器/缓冲器;

内部结构:

- (1) 数据端口 ABC: 可分为 A 组 (A+C 高 4 位) 和 B 组 (B+C 低 4 位) 两个控制组。C 的高/低 4 位端口分别为 A 和 B 输出控制信号和输入状态信号。A 和 B 可同步/异步方式工作, 同步方式下, C 为输入输出接口; 异步方式下, 为 A 和 B 的联络信号。
- (2) A/B 组控制部件: 接受数据总线的控制字; 接受读/写命令。

2. 控制字格式

D7=1 (方式选择控制字)

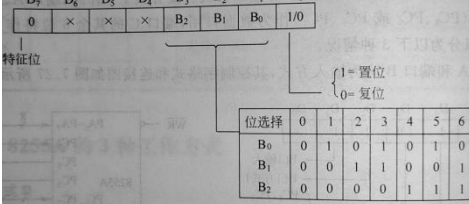
D4: A 口 I/O; D1: B 口 I/O; D3/D0: C 口高/低 4 位 I/O;

(对于 I/O: 1=输入, 0=输出)

D6D5: 选择 A 口工作方式 (00, 01, 1X 对应方式 0, 1, 2);

D2: 选择 B 口工作方式 (0, 1 对应方式 0 和 1)

D7=0 (C 口置位/复位控制字)



3. 工作方式

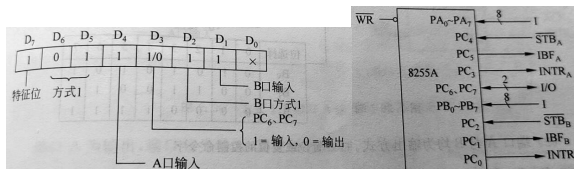
方式 0——基本输入/输出

- (1) 可同步传送或程序查询传送。查询传送时, 要有应答信号, 通常将 AB 作为数据端口, C 的 4 位作为控制信号输出, 其他 4 位作为状态输入口;
- (2) 任何一个端口都可输入或输出。输出时, 3 个口都有锁存功能; 输入时, 只有 A 有锁存, B 和 C 只有缓冲;
- (3) 由 A, B, C 高 4 位与 C 口低 4 位 4 组可组合成 16 种不同的输入/输出组合。这 4 个独立的并口只能按 8 位 (A、B) 或 4 位 (C 高、低) 作为一组同时输入/输出, 不可把其中一部分位作为输入, 另一部分位作为输出。同时, 是一种单向传送, 一次初始化不能指定端口既输入又输出;
- (4) 不设专用联络信号线, 若需要联络由用户任意指定 C 中

某一位完成联络功能。

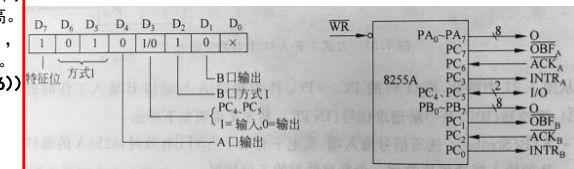
方式 1——选通输入/输出工作方式 (异步)

A、B (8 位) 及 C (2 位 PC4、PC5 或 PC6、PC7) 作数据口 (在联络信号控制下才能 I/O 操作), C 的其余 6 位作控制口使用。



(一) A, B 均为输入方式

- (1) STB*: 选通信号输入端, 由外设提供供给 8255A, =0 时, 外部数据经 PA/B7~PA/B0 送入 8255A 的输入缓冲器;
- (2) IBF: 输入缓冲器满信号 (输出), 是 STB* 的有效响应信号, 表示外设的数据已装入 A/B 缓冲器中, CPU 完成读入操作后由 RD* 上升沿使之复位 (变低电平);
- (3) INTR: 中断请求信号 (输出) 当 STB*, INTE, IBF 全为高时 INTR 有效, 向 CPU 申请中断, 表示数据端已有一个新数据; 若响应则读入改数据, 并由 RD* 下降沿来复位 (变低电平);
- (4) INTE: 中断允许信号 (无输入/输出功能), =1 表示允许; 由 A/B 的 INTR 通过对相应的 STB* 置位 1 或复位 0 来控制。



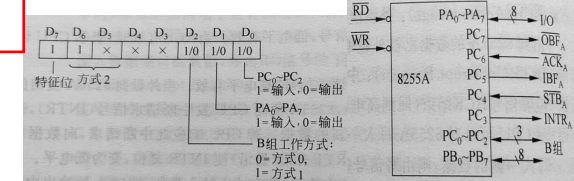
(二) A, B 均为输出方式

- (1) OBF*: 输出缓冲器满信号 (输出), 表示 CPU 向外设输出的数据已放入相应输出缓冲器, 此时 WR* 上升沿使之变为低电平;
- (2) ACK*: 外设应答信号 (输入), =0 时表明数据已被外设取走, 同时 ACK* 使 OBF* 复位 (变高电平);
- (3) INTR: 中断请求信号 (输出) 当 ACK* 结束 (高电平) 时有效, 向 CPU 发送中断请求, 表明可以对 8255A 写入新数据, 若响应则由 WR* 上升沿来复位 (变低电平);
- (4) INTE: 中断允许信号, =1 表示允许; 由 A/B 的 INTR 通过对相应的 ACK* 置位或复位来控制。

(三) 混合输入与输出

注意控制字格式中的方式选择的变换和 C 的控制口变化

方式 2——选通双向传输 (异步且仅适用于端口 A)



- (1) 方式 2 中 A 可在某时刻输入, 在另一时刻又可输出;
 - (2) 需要有两组 6 位联络信号, 分别与输入和输出设备进行联络, 联络信号由 C 中的 5 个位置 (PC7~PC3) 提供, 其中输入和输出的中断请求信号合并为一个 PC3;
 - (3) 联络信号由 C 提供, C 只有 8 位, 只能为一个端口提供联络在方式 2 下工作, 故仅适用于 A 组控制的 A 端口。
- INTR A: A 完成一次输入/输出后, 可通过其向 CPU 发中断请求; STB A: =0 时把外设数据锁存入 A;

INTE1/2: A 的输出/输入中断允许信号 (PC6/4 控制)

4. 编程应用 (初始化)

MOV AL, 82H; 初始化, A 口输出, B 口输入, 均工作在方式 0

OUT 83H, AL; 83H 为控制字地址

MOV AL, 00H

OUT 80H, AL; 80H 为 A 端口地址, 使 PA3~PA0 均等于 0

程序: ; 判断是否有键按下

```
MOV AL, 82H; 初始化8255 A口输出, B口输入, 方式0
OUT 83H, AL
MOV AL, 00H
OUT 80H, AL; 使A口输出全为低电平
; 此断程序为第一次扫描, 判断是否有低电平, 并排除抖动
LOOA: IN AL, 81H;
AND AL, 0FH
CMP AL, 0FH
JZ LOOA
CALL D20MS
IN AL, 81H
AND AL, 0FH
CMP AL, 0FH
JZ LOOA ; 如果不为0, 则确认有键按下, 转入STRAT
```

; 判哪个键按下 (确定行号)

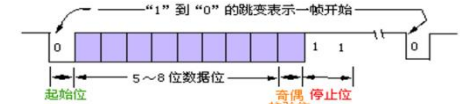
```
START: MOV BL, 4; 行数
MOV BH, 4; 列数
MOV AL, 0FEH; 先扫描0行
MOV CL, 0FH; 键盘屏蔽码
MOV CH, 0FFH; 起始键号
OUT 80H, AL; 扫描一行
ROL AL, 1; 修改得到下一行扫描码
MOV AH, AL; 保存该扫描码
IN AL, 81H; 读列值
AND AL, CL; 屏蔽高四位
CMP AL, CL; 比较
JNZ LOP2; 有列值为0, 转LOP2, 找列线
ADD CH, BH; 无键按下, 修改键号
MOV AL, AH; 恢复扫描码
DEC BL; 行数减1
JNZ LOP1; 未扫完, 转LOP1
JMP START; 重新扫描
```


7.4 可编程串行异步通信接口芯片 8250（全双工）

1. 串行通信规程:

串行通信方式

同步:发送和接收端为同步时钟来制。字符会组成一个个信息组，即信息帧。每一帧开始是同步字符，之后信息字符挨个传输。在没有信息传输时，要填充空白符，字符之间不允许有间隙。
异步:每个字符前后要加上一些标志位组成一帧信息。标志位用作帧与帧之间的分隔位。故两个字符之间的传输间隔是任意的。接收和发送的时钟频率不必完全一样，只要相近即可。



数据位：低位在前，高位在后
奇偶校验位：数据位加上校验位一起，1的个数为奇/偶数

2. 结构与工作原理

基本功能:

全双工异步通信接口芯片
通信波特率：50~9600 波特每秒
每个字符可以传送 5~8 位
可进行奇偶校验
引脚及其功能:

(1) 与 CPU 接口的信号线:

A. 数据线: D7~D0

B. 读/写控制信号线

- CS0、CS1、CS2*: 片选;
- DISTR, DISTR*: 数据输入选通，任一有效 CPU 可读出信息 (DISTR*连系统总线上的 IOR*)；
- DOSTR, DOSTR*: 数据输出选通，任一有效 CPU 可写入命令 (DOSTR*连系统总线上的 IOW*)；
- A2、A1、A0: 地址选择线，用于选择内部寄存器；
- ADS*: 地址锁存输入引脚，=0 选通地址和片选信号，=1 锁存 (ADS*接地)；

C. 总线驱动器控制线

- CSOUT:=1 表示片选信号均有效；
- DDIS: 禁止驱动器输出，CPU 读入时=0，非读入时=1；

D. 复位信号输入线: MR (接 RESET)

E. 中断信号线: INTPRT (接收出错，接收数据寄存器满，发送数据寄存器空，Modem 状态改变 4 种中断均能产生)

(2) 与通信设备接口的信号线:

A. 串行数据 I/O 线: SIN: 串行数据输入 SOUT: 串行数据输出

B. 联络控制线 (外设为 Modem 或数据装置)

- CTS*: 清除发送信号线 (输入)，=0 时表示发送结束，允许 8250 向外设发新数据，是外设对 RTS* 的应答信号；
- RTS*: 请求发送 (输出)；
- DTR*: 数据终端就绪就绪 (输出)，表示已准备好通信；
- DSR*: 数据装置准备好 (输入)，表示已与外设建立通信；
- RLSD*: 载波检测 (输入)，表示外设已检测到通信线路的信息，与 RS-232 的 DCD* 信号 (调制解调器检测到信号) 对应；
- RI*: 振铃指示，表示外设已接收到电话线上的振铃信号；

C. 用户编程端口: OUT1*, OUT2* 均为用户指定的输出引脚

D. 时钟信号线

- BAUDOUT*: 波特率信号 (输出)，XTAL1 分频后输出，频率为发送数据波特率的 16 倍，若接到 RCLK 上可作接收时钟；
- RCLK: 接受时钟 (输入)，通常与 BAUDOUT* 相连；
- XTAL1, XTAL2: 时钟信号输入和输出，可在二者之间接晶体振荡器 (1.8432MHz)；

3. 内部结构及内部控制状态寄存器功能

输入输出部分

接收数据缓冲寄存器 RBR (IN, 3F8H): 存放接收的第一个字符；
发送数据保持寄存器 THR (OUT, 3F8H): CPU 将待发送的字符写入该寄存器中，第 0 位是串行发送的第 1 位数据；

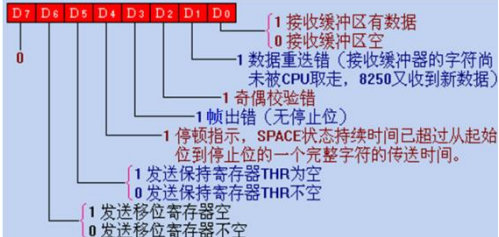
通信线路控制寄存器 LCR (OUT, 3FBH):

规定了 **异步串行通信的数据格式**；



通信线路状态寄存器 LSR (IN, 3FDH):

向 CPU 提供有关 **数据传输的状态信息**；



波特率因子寄存器 DLRL (OUT, 3F8H-L, 3F9H-H):

8250 芯片规定当通信线路控制寄存器写入 D7=1 时，接着对地址 3F8H、3F9H 可分别写入波特率因子的低字节和高字节；
波特率因子 = (8250 晶振时钟频率) 1843200 / (16 × 波特率)

表 7.6 波特率和除数对照值					
十进制	十六进制	波特率	十进制	十六进制	波特率
1047	417	110	96	60	1200
768	300	150	48	30	2400
384	180	300	24	18	4800
192	C0	600	12	0C	9600

中断控制部分

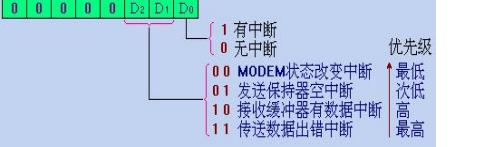
中断允许寄存器 IER (OUT, 3F9H): 低 4 位允许 8250 4 种类型中断 (相应位置 1)，并通过 IRQ4 向 8088CPU 发中断请求；



优先级: D2=1 (奇偶错, 帧错, 停顿等) > D0=1 > D1 > D3=1；

中断标识寄存器 IIR (IN, 3FAH): 可判断中断的有无和类别；

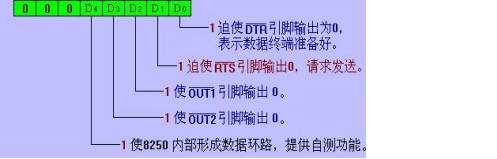
注: 可产生 4 种, 但对 INTPRT 只提供优先级最高的一个



与 Modem (调制解调器) 有关

Modem 控制寄存器 MCR (OUT, 3FCH):

设置联络线，控制与调制解调器或数传机的接口信号；



Modem 状态寄存器 MSR (IN, 3FEH): 反映了 Modem 控制线的当前状态，提供了低 4 位记录输入信号变化的状态信息；



4. 寻址方式 (地址线 A2A1A0 寻址 8250 内部的 10 个寄存器)
波特率因子寄存器 (高低字节) 与收发缓冲器以及中断允许寄存器地址相重，由线路控制寄存器的 D7 位来指定。

- 000—THR (写信号选择) /RBR (读信号选择) 且 D7=0
- 001—IER, 且 D7=0
- 010—IIR 011—LCR 100—MCR 101—LSR 110—MSR
- 000—DLR (L), 且 D7=1
- 001—DLR (H), 且 D7=1

5. 编程应用

初始化:

(1) 设置波特率。
例如，设波特率为 9600，则波特率因子 N=12
MOV DX, 3FBH;
MOV AL, 80H; 通过线路控制寄存器 D7=1，设置波特率
OUT DX, AL
MOV DX, 3F8H
MOV AL, 12
OUT DX, AL; 写波特因子寄存器低字节
INC DX
MOV AL, 0
OUT DX, AL; 写波特因子寄存器高字节，3F9H 送 0

(2) 设置串行通信数据格式
例如，数据格式为 8 位，1 位停止位，奇校验。
MOV AL, 0BH
MOV DX, 3FBH
OUT DX, AL

(3) 设置工作方式
无中断: MOV AL, 3 ; DTR、RTS 有效, OUT1/2, LOOP 位无效
MOV DX, 3FCH
OUT DX, AL

有中断: MOV AL, 0BH ; OUT2=0, 开中断
MOV DX, 3FCH
OUT DX, AL

循环测试: MOV AL, 13H ; 自发自收
MOV DX, 3FCH
OUT DX, AL

2. 程序查询方式通信程序

发送程序: 读 LSR (3FDH) 中 D5 位, 检查发送保持器是否空 (D5=1)
TR: MOV DX, 3FDH
IN AL, DX
TEST AL, 20H
JZ TR ; 发送保持器不空 (D5=0), 转 TR
MOV AL, [SI] ; 发送保持器空, 发下一数据
MOV DX, 3F8H
OUT DX, AL

接收程序: 读 LSR (3FDH) 中 D0 位, 检查接收数据寄存器是否就绪 (D0=1)
RE: MOV DX, 3FDH
IN AL, DX
TEST AL, 1
JZ RE ; 接收缓冲器不满 (D0=0), 转 RE
MOV DX, 3F8H ; 接收缓冲器满, 读数据
IN AL, DX
MOV [DI], AL

3. 中断方式编程

用 8250 中断方式进行通信编程需要完成:

- (1) 对 8259A 中断控制器进行初始化，允许中断优先级 4

MOV AL, 13H; 单片使用，需要 ICW4
MOV DX, 20H
OUT DX, AL; ICW1
MOV AL, 8; 中断向量号为 08H~0FH
INC DX
OUT DX, AL; ICW2
INC AL; 缓冲方式，8088/8086
OUT DX, AL; ICW4
MOV AL, 8CH; 允许 0, 1, 4, 5, 6 级中断
OUT DX, AL; 送屏蔽字 OCW1
(2) 设置中断向量 IRQ4. IRQ4 中断类型号为 0CH, 0CH*4=30H
设中断服务程序入口地址为 2000: 100
XOR AX, AX
MOV DS, AX
MOV AX, 100H
MOV WORD PTR [0030H], AX; 送 100H 到 00030H 和 00031H 中
MOV AX, 200H
MOV WORD PTR [0032H], AX; 送 2000H 到 00032H 和 00033H 中
(3) 对 8250 送中断允许寄存器 (3F9H) 设置允许/屏蔽位
例: 允许发送与接收中断请求
MOV AL, 3
MOV DX, 3F9H
OUT DX, AL
(4) 中断结束返回时，对 8259A 发 EOI 命令
MOV AL, 20H
MOV DX, 20H
OUT DX, AL; 发 EOI 命令, OCW2
IRET; 开中断允许，并返回

8.2 80386 微处理器

一、80386 微处理器 简介

- 80386 是第一个 **全 32 位** 微处理器，简称 1-32 系统结构。
- 其 **数据总线** 和内部数据通道都是 **32 位**，能灵活处理 **8 位、16 位、32 位 3 种数据类型**。
- 其 **地址总线** 也是 **32 位**，能寻址 **2³² 字节 (4GB)** 的物理存储空间。
- 80386 的逻辑存储器采用 **分段结构**，一个段最大可达 **4GB**。
- 其运算速度比 80286 快 3 倍以上。

二、80386 的特点

- 80386 芯片在硬件结构上由 **6 个逻辑单元** 组成。它们按 **流水线** 方式工作，运行速度可达到 **4 MIPS**。
- 80386 有三种方式: 实方式、**保护方式**、**V86 方式**。
- 支持 **多用户、多任务 (主要在保护模式下)** 一条指令就可以完成任务切换。
- 支持存储器的 **分段和分页管理**，可以实现 **虚拟存储系统**。
 - 多用户、多任务需要 **各个用户、各个任务拥有自己的存储空间**，相互之间完全 **隔离**，起到对用户和任务的保护作用，这一点要依靠 **巨大的虚拟存储空间** 支持。
 - 保护方式下，运用虚拟存储管理，能寻址 **2⁴⁶ 字节 (64TB)** 的 **虚拟存储空间**。
- 多用户、多任务的保护机制:
- 特权保护机制**
 - 4 级特权级**: 0 级最高，其次为 1、2 和 3 级。
 - 一般 **0、1 和 2 级** 用于 **操作系统程序**，**3 级** 用于 **用户程序**。
 - 除 **特权级检查** 外，每条指令执行期间，CPU 还要进行 **访问类型、内存超限等保护特性检查**。
 - 通过 **虚拟存储管理**，在用户与用户之间、用户和操作系统之间形成严格的 **隔离保护**。

三种工作模式

一、实地址模式 (实模式)

当 80386 在 **刚加电启动或复位** 时，便由操作系统自动控制进入 **实模式**。

实模式 主要是为 80386 进行 **初始化** 用的，为 80386 保护模式所需要的数据结构做好各种配置和准备。

在实模式下，80386 类似于 8086 的体系结构，其寻址方式、中断处理方式等都 **与 8086 兼容**。

从编程角度看:

- 可以用 32 位寄存器进行编程，加快了执行速度；
- 增加了两个段寄存器 FS 和 GS，可以访问的段达到 6 个；
- 新增指令可简化一些操作。

二、保护虚拟地址模式 (保护模式)

保护模式是80386最常用的、也是最具特色的工作模式。

通常在开机或复位后, 先进入实模式完成初始化, 然后就立即切换到保护模式。

保护模式提供了多任务环境中的各种复杂功能以及对复杂存储器组织的管理机制:

- 1) 多用户、多任务;
- 2) 虚拟存储管理;
- 3) 保护机制

> 特权保护机制

> 不同任务之间的隔离(不同的虚拟存储空间)

> 虚拟存储空间: 是指程序所占有的存储空间。

80386系统中, 由于内存容量的限制, 不可能将所有的段都放在内存中, 而必须将大部分段放在外部磁盘上, 待执行到相关程序段时再调入内存, 将暂时不执行的程序段调出内存。

程序员编写程序时, 其程序存入磁盘。这样, 从程序员的角度来看, 系统中似乎有一个容量很大、速度也相当快的虚拟存储器; 其实, 它并不是真正物理上的内存。

386物理存储空间: $2^{32}=4GB$;

虚拟存储空间有多大? $2^{46}=64TB$ (???)

三、虚拟地址8086方式 (V86方式)

为了解决80286中不能在保护模式下运行8086/8088应用程序的问题, 从80386开始, 在保护模式中引入了虚拟8086工作模式 (简称V86模式)。

V86模式是让80386模拟8086 1MB空间的寻址环境, 但它并不仅限于1MB的存储空间, 因为它可以同时支持多个V86环境。

V86模式下, 多个8086实模式的应用软件可以同时运行。

V86模式的主要特点

1. V86模式的寻址

在V86模式下, 80386对段寄存器的运用和实模式一样。

2. V86模式下的保护功能

V86模式具有80386的一些保护功能, 但只是简单地把V86模式下任务的特权级一律降到最低级3。

3. V86模式下的分页功能

为了使80386运行多个虚拟任务, 地址空间大于1MB, 需要启动分页机制, 必须预先在保护模式下对CR₀的PG位 (位31) 置1。

四、80386 三种工作模式的切换

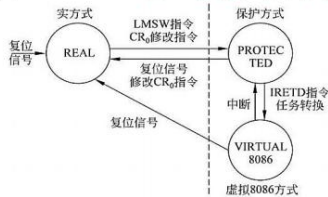


图 8.13 80386 的 3 种工作方式的相互转换示意图

CPU复位后进入实地址模式;

CR₀的PE位置1, 可以使CPU从实地址模式切换到保护模式;

EFLAG的VM位置1, 可使CPU从保护模式转变到V86模式。

实模式不能切换到V86模式。

(二) 6个段寄存器 (16位)

386工作在实方式:

16位段寄存器里存放的还是段基址(段起始地址的高16位)

段寄存器的使用、存储器物理地址的计算类似于8086

CS	段基址
SS	段基址
DS	段基址
ES	段基址
FS	段基址
GS	段基址

增加了2个数据段寄存器

(二) 6个段寄存器 (16位)

386工作在保护方式:

16位段寄存器不再直接存放段基址, 而是存放一个选择子 (selector), 通过它查表得到段描述符, 进而得到段基址、段界限、段属性。

16位的段寄存器	段描述符高速缓存 (64位, 自动加载)	
CS	段选择子	段基址地址
SS	段选择子	段基址地址
DS	段选择子	段基址地址
ES	段选择子	段基址地址
FS	段选择子	段基址地址
GS	段选择子	段基址地址

为加快查表和计算, CPU里每个选择子有对应的描述符高速缓存。

描述符和描述符表

> 描述符:

为了实现内存分段管理和保护功能, 每个段都有一个描述符; 段描述符中含有对应段的基址、段界限和段的属性等信息。各种段描述符由操作系统建立, 并组成描述符表, 放在内存中。

> 描述符表:

1) 全局描述符表(Global Descriptor Table, GDT)

系统只有一个GDT

2) 局部描述符表(Local Descriptor Table, LDT)

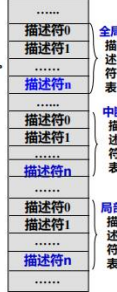
每个任务都可以有自己的LDT

系统可以有多个LDT

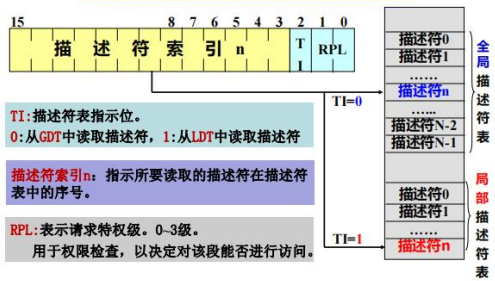
3) 中断描述符表(Interrupt Descriptor Table, IDT)

每个中断都有一个中断描述符, 类似于中断向量

系统只有一个IDT



用选择子 (selector) 查表



怎么在内存中找到这些描述符表呢?

(三) 地址寄存器和系统段寄存器

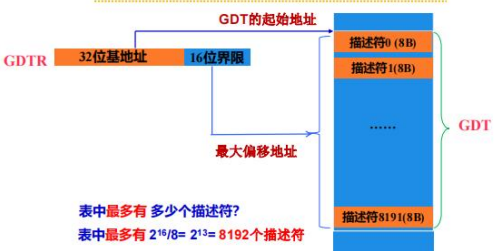
GDTR 48位	32位线性基地址	16位界限值
IDTR 48位	32位线性基地址	16位界限值

LDTR	16位选择子	段基址地址	段界限和属性
TR	16位选择子	段基址地址	段界限和属性

GDT: 全局描述符表
IDT: 中断描述符表
LDT: 局部描述符表
TSS: 任务状态段

GDTR: 全局描述符表寄存器
IDTR: 中断描述符表寄存器
LDTR: 局部描述符表寄存器
TR: 任务寄存器

GDTR与GDT的关系



例1. 如果GDTR的内容为200000001FFFh, 请给出GDT的起始地址、结束地址、表的长度。表中放了多少个描述符? 最后一个描述符的地址范围?

解: GDT的起始地址 = 20000000h

GDT的结束地址 = 20000000h + 1FFFh = 20001FFFh

GDT表长 = 1FFFh + 1 = 2000h

表中描述符个数 = 2000h ÷ 8 = 400h = 1024

最后一个描述符的地址范围: 20001FFFh - 7 = 20001FF8h ~ 20001FFFh

习题 9.5:

如果GDTR的内容为002100001FFFh, 请给出GDT表的起始地址、结束地址、表的长度。

表中放了多少个描述符? 第5个描述符的地址范围?

答: GDTR = 0021 0000 01FFFh

表起始地址 = 0021 0000h

表结束地址 = 0021 01FFFh

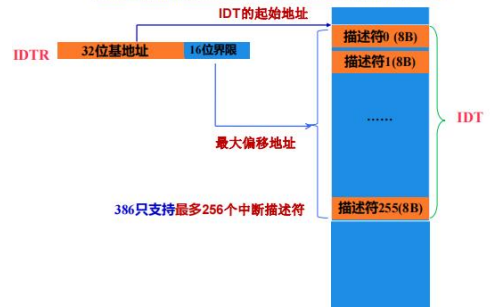
表长 = 01FFFh + 1 = 0200h

描述符个数 = 0200h ÷ 8 = 40h = 64

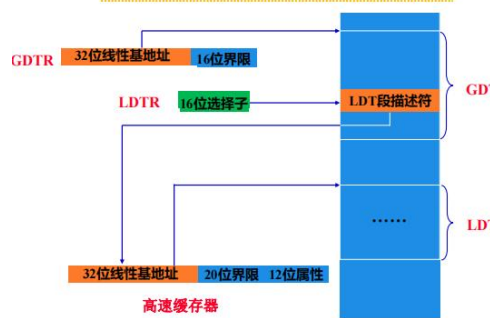
第5个描述符的地址范围:

0021 0000h + 5 * 8 = 00210028h ~ 0021002Fh

IDTR与IDT的关系



LDTR、LDT与GDT的关系



例2. 假设GDT的基地址为01002000h, LDTR = 2108h, 问LDT描述符的地址范围?

解: LDTR = 0010000100001000

TI = 0, LDT描述符在GDT中

索引 = 00100001000001

LDT描述符的起始地址:

GDT的基址地址 + 索引 × 8

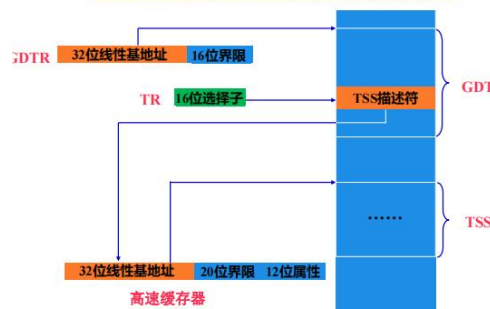
= 01002000h + 00100001000001 × 8

= 01002000h + 2108h = 01004108h

LDT描述符的地址范围 (占8字节):

01004108h ~ 0100410Fh

TR、TSS与GDT的关系



例3. 假设GDT的基地址为00011000h, LDTR = 3209h, 问LDT描述符的地址范围?

答: LDTR = 0011001000001001

TI = 0, LDT描述符在GDT中

索引 = 00110010000001

LDT描述符的起始地址 = GDT的基址地址 + 索引 × 8

= 00011000h + 0011001000001 × 8 = 00014208h

LDT描述符的地址范围 (占8字节):

00014208h ~ 0001420Fh

(三) 指令指针和标志寄存器

32位的指令指针 (EIP)
32位标志寄存器 (EFLAGS)

实模式时采用16位的指令指针IP (EIP低16位)。

EFLAGS的低12位与8086标志寄存器FLAGS完全相同。
高20位中设置了5位新的标志。
VM: 1—V86方式, NT: 1—任务嵌套

(四) 控制寄存器

32位的控制寄存器CR0、CR2和CR3, 保存各种全局性状态。

CR0是最常用的一个控制寄存器, 选择工作模式。

CR3为页目录基址寄存器, 存放页目录表的物理基地址。

CR0: PG ET TS EM MP PE

PG	PE	80386的工作模式选择
0	0	实模式
0	1	分段不分页的保护模式
1	1	分段且分页的保护模式

(一) 实模式

80386的所有指令在实模式下都有效。其物理地址的形成与8086相同, 可寻址的**实地址空间**只有**1MB**, 段的**最大长度为64KB**。

中断向量表仍设置在**00000H~003FFH**共计**1K**字节的存储区内。

系统**初始化区**在**FFFFFF0H~FFFFFFFH**存储区内。

(二) 保护模式

保护模式下, 80386采用**段页式**存储管理机制, 利用片内存储管理单元 (MMU)来实现对存储器的**两级管理**:

分段管理 (逻辑地址→线性地址)

分页管理 (线性地址→物理地址) (分页是**可选的**)

在两级存储管理中, **段的大小可以选择** (以**字节**或**页**为单位), 根据数据结构和代码模块的大小而定。

另外, 对每个段还可以赋予**访问权**和**保护信息**, 以有效防止在**多任务环境**下各个任务对存储器的越权访问。

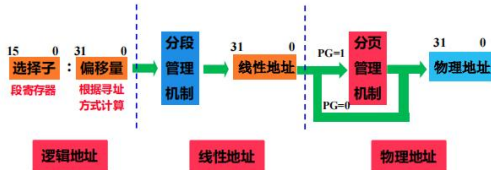
➤ **分段管理**: 实现对**逻辑地址空间**的管理。

把由**指令**指定的**逻辑地址(虚拟地址)**变换成**线性地址**, **段的大小可选择以字节或以页为单位**, 根据数据和代码模块的大小而定。

➤ **分页管理**: 实现对**物理地址空间**的管理。

- 把由分段管理产生的**线性地址**变换成**物理地址**。
有了物理地址后, 才能访问存储器或输入/输出端口。
- 分页单元将一个段转换为多个页面, **每页大小固定为4K字节** (磁盘一个扇区的大小)。
 - 在**内存和磁盘**之间进行**地址映射**时, 系统都是以**页为单位**把磁盘的程序和数据转存到内存中一个相对应的地址区间。
 - 分页是可选的**
如果不使用**分页**, **线性地址就是物理地址**。

保护模式下的段页式存储管理



注意区分**逻辑地址**、**线性地址**和**物理地址**这3者之间的关系。

保护模式下的地址转换

1) 分段: **逻辑地址** 转换为 **线性地址**

逻辑地址由**段寄存器**和**偏移地址**组成

- 段寄存器中存放的不再是段基址, 而是**选择子**, 通过**选择子**来选择描述符表中的**描述符**, 从描述符中得到**段基址**, 再将**段基址**和**偏移地址**相加即可得到**线性地址**;

2) 分页: **线性地址** 转换为 **物理地址**

分页可选

如果**不分页**, 物理地址=线性地址

➤ **逻辑地址(虚拟地址)**:

编程时

16位 段选择子(段寄存器) + 32位 偏移地址
CS--- EIP
SS--- ESP
DS,ES,FS,GS
--- 根据寻址方式计算

➤ **线性地址**:

由存储器**分段管理机制**按下式来计算:

线性地址 = 32位 段基址(查描述符得到) + 32位 偏移地址

➤ **物理地址**:

不分页时, 物理地址 = 线性地址;

分页时, 由存储器分页管理机制转换: 物理地址 = F(线性地址)

描述符与描述符表

为了实现分段管理, 80386把有关段的信息存放在一个称为**段描述符** (简称描述符, **8个字节**)的数据结构中, 并把系统中所有的描述符按类组成一张张**描述符表**, 以便硬件查找和识别。

80386共设置了**3种**描述符表:

- 全局描述符表 GDT**
- 局部描述符表 LDT**
- 中断描述符表 IDT**

➤ **两种类型的段**: **非系统段**、**系统段**

➤ **两种段描述符**: **非系统段描述符**

系统段描述符

➤ **非系统段**: **代码段**、**数据段**、**堆栈段**

➤ **系统段**: **LDT, TSS**

各种门--- 任务门, 调用门, 中断门等

段描述符包括:

- 段基址 (Base Address)**, 规定了线性地址空间中段的起始地址。也可以把基址看成是段内偏移量为0的线性地址。
- 段的界限 (Limit)**, 表示在虚拟地址中, 段内可使用的最大偏移量。
- 段的属性 (Attributes)**, 包括该段是否可读、写入以及段的特权级等, 也叫**访问权字节**。

非系统段描述符格式

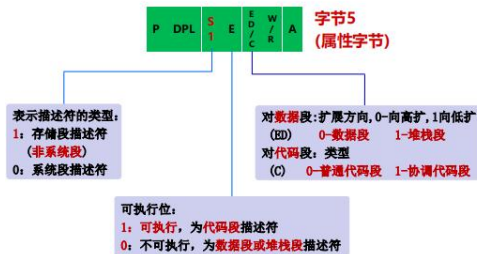


属性字节 (访问权字节) (8位): B5

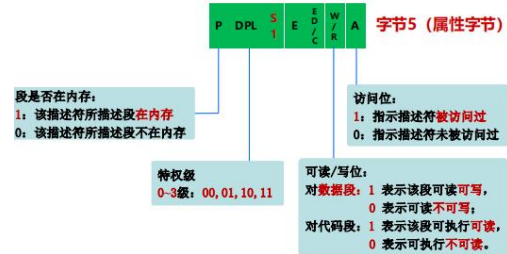
基地址 (32位): B7 B4 B3 B2

界限 (20位): B6 (低4位) B1 B0

非系统段描述符格式



非系统段描述符格式



非系统段的段界限

段界限决定段的寻址范围, 它由描述符中20位段限长及**G**位得到。

字节6 G D 0 A V L 段界限值 19~16位 字节1 段界限值的15~0位 字节0

段界限粒度位:
0: 界限粒度为**字节**; 1: 界限粒度为**页(4K字节)**

- G=0:** 32位段限长的高12位为0, 低20位是段描述符中**20位段界限值**, **段最大寻址范围为1MB**。
- G=1:** 段限长 = 描述符中规定的**20位段限长左移12位 + 0FFFFH**, **段最大寻址范围为4GB**

非系统段的段界限

段基址为20000000H, 段描述符中的20位限长为0FFFFFH。当描述符中的**G=0**时, 此段的范围和段长?

G=0:
段的范围: 20000000H-200FFFFFFFH
段长: 1MB

段基址为00000000H, 段描述符中的20位限长为0FFFFFH。当描述符中的**G=1**时, 此段的范围和段长?

G=1:
段界限为: 0FFFFFFFH左移12位+0FFFFH = 0FFFFFFFHH
段的范围: 00000000H-0FFFFFFFHH
段长: 4GB

例 请分析下面的描述符:

80	00	6
B2	0A	4
00	00	2
03	FF	0

解: **属性字节**: B2h = 10110010
P DPL S ED W A

S=1, 非系统段; E=0, ED=0, 数据段; W=1, 可写; DPL=1; P=1, 在内存; A=0, 未访问过。

段基址: 800A0000h **界限**: 003FFh **段长**: 3FFh+1=400H

段在内存的地址范围: 800A0000h~800A03FFh

系统段描述符 (S位为0)

系统段包括局部描述符表**LDT**、任务状态段**TSS**和各种**门**。



LDT描述符



例 请分析下面的描述符：

00	00	6
A2	10	4
00	00	2
00	FF	0

解：属性字节：A2H=1 0 1 0 0 0 1 0

P DPL S TYPE

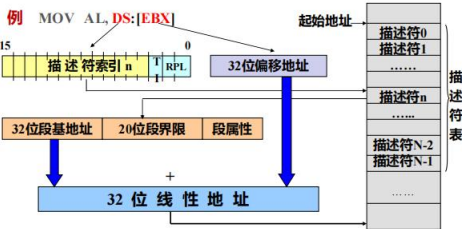
S=0, 系统段；TYPE=2, LDT描述符；

DPL=1; P=1, 描述符有效。

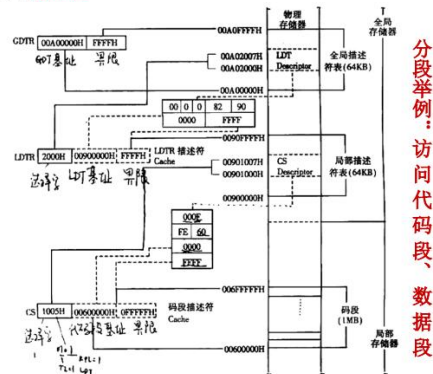
LDT基址：00100000h 界限：000FFh 表长：0FFh+1=100H

LDT在内存的地址范围：00100000h~001000FFh

分段管理 (逻辑地址→线性地址)



将段选择子指向的段描述符中32位基址与32位偏移地址相加，即得线性地址。



80386工作在保护模式下，访问代码段、数据段，请回答下面问题：

1) GDTR=00A00000FFFFh

GDT的起始地址 = ? GDT的结束地址 = ?

答：GDT的起始地址 = 00A00000h

GDT的结束地址 = 00A00000h+FFFFh = 00A0FFFFh

2) LDTR = 2000h

LDT描述符的地址范围（占8字节）：？

答：LDTR = 2000h = 0010000000000000h

TI = 0, LDT描述符在GDT中，索引 = 001000000000

LDT描述符的起始地址：

GDT起始地址+索引×8= 00A00000h+2000h= 00A02000h

LDT描述符的地址范围(占8字节)：00A02000h ~ 00A02007h

3) LDT描述符：0000, 8290, 0000, FFFFh

LDT基址 = ?，界限 = ?

答：LDT基址 = 00900000h，界限 = 0FFFFh。

4) CS = 1005h = 0001000000000101b

代码段描述符的地址范围（占8字节）：？

答：CS = 1005h = 0001000000000101b

TI = 1, 代码段描述符在LDT中，索引 = 000100000000

代码段描述符的起始地址：

LDT起始地址+索引×8= 00900000h+1000h= 00901000h

代码段描述符的地址范围(占8字节)：00901000h ~ 00901007h

5) 代码段描述符：000F, FA60, 0000, FFFFh

代码段基址 = ?，界限 = ? 分析属性：？

答：代码段基址 = 00600000h，界限 = 0FFFFh。

属性：FAh = 11111010b

代码段，可读，DPL=3，在内存中，未访问过。

6) 假设EIP = 00012000h，CR0的PG位= 0（不分页）

则当前要取的指令在内存中的物理地址（起始的地址）=？

答：当前要取的指令在内存中的物理地址（起始的地址）

= 代码段基址+偏移地址(EIP)

= 00600000h+00012000h = 00612000h

注：如果CR0的PG位= 1（分页），则上面求出的00612000h就不是物理地址，而是线性地址，需要进一步分页转换。

7) 现在要访问一个数据段

数据段描述符为：0001, F220, 0000, FFFFh

该数据段 基址 = ? 界限 = ?，分析属性：？

答：该数据段 基址 = 00200000h，界限 = 1FFFFh

属性：F2h = 11110010b

数据段，可写，DPL = 3，在内存，未访问过。

8) 如果在代码段中执行下面一段程序，程序中访问的数据段描述符如7)所示。

MOV AX, 1005H ;访问数据段的选子

MOV DS, AX

MOV EBX, 55667788H

MOV EAX, [EBX] ;基址寻址

如果 CR0的PG位= 0（不分页）

那么，要读入EAX中的数据在内存的物理地址范围(4B)=？

答：要读入EAX中的数据在内存的物理地址范围(4B)

= 数据段基址+偏移地址

= 00200000h+55667788h = 55867788h ~ 5586778Bh

9) 如果在代码段中执行下面一段程序，程序中访问的数据段描述符如7)所示。

MOV AX, 1005H ;访问数据段的选子

MOV DS, AX

MOV EBX, 55667788H

MOV ESI, 10000000H

MOV EAX, [EBX+ESI] ;基址加变址寻址

如果 CR0的PG位= 0（不分页），

那么，要读入EAX中的数据在内存的物理地址范围(4B)=？

答：要读入EAX中的数据在内存的物理地址范围(4B)

= 数据段基址+偏移地址

= 00200000h+(55667788h+10000000h)

= 65867788h ~ 6586778Bh

分页管理

当CR₀中的PG=1时，80386系统就启动分页机制。

分页管理的对象是4KB的存储块，称之为页。

分页采用了两级表结构：页目录表和页表

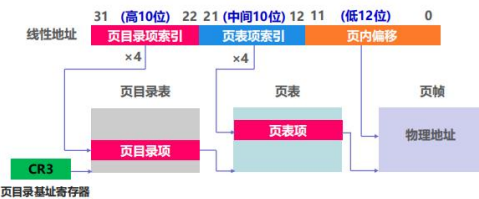
分页机制实现两级地址转换：

➢ 在较高一级，由页目录表中的页目录项(页目录描述符)映射页表；

➢ 在低一级，由页表中的页表项(页描述符)映射页帧。

➢ 页目录表、页表和页帧都是按页存放(4KB)，在内存的起始地址的低12位为0。

线性地址到物理地址的转换



- CR₃：高20位为页目录表32位基址的高20位，表基址低12位为0。
- 页目录项索引：线性地址高10位(31~22位)，查页目录表的索引。
- 页表项索引：线性地址中间10位(21~12位)，查页表的索引。
- 页内偏移量：线性地址低12位(11~0)，页帧内的偏移地址。

页目录项(页目录描述符)和页表项(页描述符)



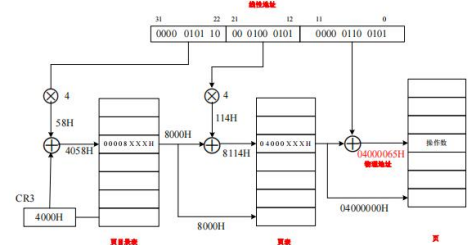
当页目录项和页表项的U、W位不一致时，选取最小的值。

例 假设页目录项的U W位= 11（用户可写），页表项的U W位= 10（用户可用，但只可读、不可写）

则，最终U W位= min(11, 10) = 10（用户可用，但只可读、不可写）

线性地址转换成物理地址举例

假设线性地址=05845065H，CR₃=4000H，求转换的物理地址。



页目录表：00004000h~00004FFFh，页表：00008000h~0000FFFFh，页帧：04000000~0400FFFFh

假设页目录描述符=00008005H，页描述符=04000007H

问：操作数用户是否可写？

答：页目录描述符：U=1, W=0, 不可写

页描述符：U=1, W=1, 可写

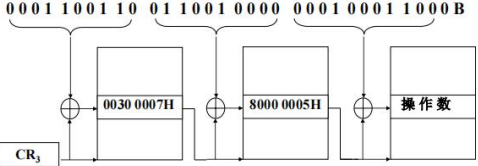
最终U W位= min(10, 11) = 10

∴ 操作数用户不可写，可读。

课堂练习 4

已知在80386系统中，CR₃=0010 0000H，页目录项和页表项内容如下图中所示，假设分段转换得到的线性地址为1999 0118H，请回答下面问题。

线性地址：



课堂练习 4 答案

1、页目录表地址范围 = 0010 0000H ~ 0010 0FFFH

2、页目录项的地址范围 = 0010 0198H ~ 0010 019BH

3、页表的地址范围 = 0030 0000H ~ 0030 0FFFH

4、页表项的地址范围 = 0030 0640H ~ 0030 0643H

5、页帧的地址范围 = 8000 0000H ~ 8000 0FFFH

6、被访问操作数(32位)的地址范围 =

8000 0118H ~ 8000 011BH

7、根据页目录项和页表项的内容，该操作数用户是否能访问？是，是否可写？否，操作数所在的页帧是否在内存？是，操作数所在的页帧是否被访问过？否。

课堂练习 5 答案

80386工作在保护方式下，GDTR = 0020 0000 3FFFh，LDTR = 3009h，CS = 1007h，EIP = 0000 0800h。

1) GDT的起始地址为 0020,0000H，

GDT的结束地址为 0020,3FFFh。

GDT的表长为 4000H。

2) LDT描述符的地址范围是 0020,3008H 到0020,300FH。

3) 若代码段描述符为：

00	00	6
FB	50	4
00	00	2
0F	FF	0

则代码段的属性为：

特权级 3，是否可读 是，

是否在内存 是，是否访问过 是。

代码段的长度为 1000H。

4) 若80386仅分段，不分页，则当前执行指令的物理地址是 0050,0800H；

若既分段，又分页，则访问代码段时转换的线性地址为 0050,0800H；

根据此线性地址进行分页转换，页目录描述符的偏移地址为 4，页描述符的偏移地址为 400H，被访问指令在页帧中的偏移地址为 800H。